BANGLA SIGN LANGUAGE RECOGNITION: DATASET INNOVATION AND METHODOLOGIES

By

1. Saad Ahmed ID: 200201008

2. Razia Sultana ID: 200201007

B.Sc. Engineering in Computer Science and Engineering

Department of Computer Science and Engineering

Faculty of Electrical and Computer Engineering

Bangladesh Army University of Science and Technology (BAUST)

JUNE 2024

Bangla Sign Language Recognition: Dataset Innovation and Methodologies

This thesis is submitted in partial fulfillment of the requirement for the degree of B.Sc. Engineering in Computer Science and Engineering

By

1. Saad Ahmed ID: 200201008

2. Razia Sultana ID: 200201007

Supervised by

Dr. S.M. Jahangir Alam

Professor and Head, Department of CSE

Co-Supervised by

Md. Khalid Syfullah

Lecturer, Department of CSE

Department of Computer Science and Engineering

Bangladesh Army University of Science and Technology (BAUST)

Saidpur Cantonment, Nilphamari, Bangladesh

The thesis titled **"Bangla Sign Language Recognition: Dataset Innovation and Methodologies"** submitted by Saad Ahmed (ID:200201008) and Razia Sultana (ID:200201007), session 2020-2021 has been presented on 24/06/2024 and accepted as satisfactory in fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering (CSE) as B.Sc. Engineering to be awarded by the Bangladesh Army University of Science and Technology (BAUST).

Board of Examiners

1. _____ Supervisor

   Dr. S.M. Jahangir Alam
   Professor and Head
   Department of Computer Science and Engineering (CSE)
   Bangladesh Army University of Science and Technology (BAUST)

2. _____ Co-Supervisor

   Md. Khalid Syfullah
   Lecturer
   Department of Computer Science and Engineering (CSE)
   Bangladesh Army University of Science and Technology (BAUST)

3. _____ Member

   Dr. Engr. Mohammed Sowket Ali
   Associate Professor
   Department of Computer Science and Engineering (CSE)
   Bangladesh Army University of Science and Technology (BAUST)

4. _____ Member

Hasan Muhammad Kafi

Assistant Professor

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

5. _____ Member

Md. Zahid Hassan

Lecturer

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

6. _____ Member

Md. Mahadi Hasan

Lecturer

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

_____

Dr. S.M. Jahangir Alam

Professor and Head

Department of Computer Science and Engineering (CSE)

Bangladesh Army University of Science and Technology (BAUST)

CANDIDATE'S DECLARATION

It is hereby declared that the contents of this thesis are original and any part of it has not been submitted elsewhere for the award of any degree or diploma.

SN.	**Student Name**	**ID**	**Signature & Date**
1.	Saad Ahmed	200201008	
2.	Razia Sultana	200201007	

ACKNOWLEDGMENTS

First and foremost, we are deeply grateful to Almighty Allah for granting us the strength and perseverance to complete this thesis successfully. We extend our sincerest thanks and heartfelt gratitude to our honorable thesis supervisor, Dr. S.M. Jahangir Alam, Professor and Head of the Department of Computer Science & Engineering (CSE) at Bangladesh Army University of Science & Technology (BAUST), Saidpur. His inspiration and unwavering support have been instrumental in the completion of our thesis.

We like to express our special thanks to the Head of the Department of Computer Science & Engineering (CSE), Dr. S.M. Jahangir Alam for his valuable suggestions, positive advice, encouragement, and sincere guidance throughout our thesis work. Additionally, we convey our gratitude to all the respected teachers of the Department of CSE, as well as those from other departments, for their support and contributions.

Our sincere thanks go to the respected Honorable Vice Chancellor, the Chief of Army Staff and board of trustees, as well as all members and academic staff who have led this institution and contributed to our bright future.

We would also like to thank all our friends and the staff of the department for their valuable suggestions and assistance. Finally, we extend our deepest appreciation to our parents and families for their unwavering love and support throughout our studies.

ABSTRACT

Bangla Sign Language (BdSL) is essential for communication among people with hearing impairments in Bangladesh. In order to address the need for improved BdSL gesture detection, this paper introduces the RSBdSL38 dataset. To ensure variety and usefulness, 11,864 gestures from 46 individuals in various difficult situations are included in the dataset. With its improvements over earlier datasets, RSBdSL38 represents a major advancement in BdSL recognition technology. The dataset's ability was tested to aid in the understanding of BdSL gestures using sophisticated machine learning algorithms. Important aspects such as accuracy and performance was examined in various scenarios and contrasted the findings with available datasets to demonstrate the power and use of RSBdSL38. The foundation for developing more inclusive and accurate BdSL recognition systems is laid by this research, which will facilitate everyday communication for BdSL users.

Keywords: Gesture Recognition, Pretrained Models, Ensemble Techniques, Robustness and Comparative Analysis.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1
Introduction

1.1 Overview

Sign language is a natural and primary form of communication for the deaf or hard of hearing. It plays an important role in their daily interactions which allowes them for subtle expression and emotional exchange. It is dependent on gestures, facial expressions and body language enabling rich and complex talks, which are essential for social bonding and cultural identification in the deaf community. Though it is efficient in the deaf and mute community, understanding and fluency in sign language can be difficult for those who are not familiar with its complexities. Efforts to raise awareness and educate people about sign language can help to overcome these gaps as well as creating greater understanding and inclusion across linguistic and cultural boundaries.

For people who are unable to talk or hear, sign language becomes the sole means of successful communication. Its universal accessibility makes it essential for people with speech and hearing difficulties, providing them with a critical lifeline for expressing their thoughts, feelings, and ideas. However, bridging communication between sign language users and those unfamiliar with its lexicon and grammar is a substantial challenge. Because there are so many different languages in the globe, sign language varies a lot from nation to nation and has unique linguistic characteristics. Though regional variants like Bengali Sign Language (BdSL) serve particular linguistic communities, American Sign Language (ASL) is an internationally recognized form of sign language [1]. The diversity in sign languages is reflective of the rich cultural and linguistic heritage of different regions. Each sign language, with its own syntax, vocabulary, and expressions, provides a unique window into the community it serves. This diversity, while beautiful, also presents challenges in creating universally accessible communication tools. For instance, a person fluent in BdSL might struggle to understand ASL without prior learning, just as a speaker of Bengali might find English difficult without

education. This gap in understanding highlights that both people who use sign language and those who do not suffer communication barriers. With the increasing use of sign language and technological improvements, there is a great chance to improve accessibility and understanding. Real-time interpretation is made possible by systems that employ computer vision and machine learning techniques. This helps to close the communication gap between people who use sign language and others. This introduces collection data specific to the grammatical nuances and cultural background of Bengali Sign Language (BdSL) shown in Figure:1.1 which shows the need for comprehensive datasets for this language [2].

Figure 1.1: Bangla Sign Language (BdSL) gestures [3].

1.2 Problem Statement

Bridging the gap between natural language and sign language is a major difficulty in the field of continuous sign language recognition. Fluent continuous sign language translation cannot be achieved without accurate word-level recognition, which is a prerequisite for sign sentences, which are composed of discrete sign words and sometimes fingerspelling for proper nouns [4]. There are two major problems that dominate the field of sign language studies. First, a basic challenge is the lack of extensive datasets customized to the complexities of sign language. Secondly, a persistent difficulty is the creation of machine learning and deep learning models that can capture all aspects of articulated sign context. Recentrly researcher has expressed need of large scale word-level sign language recognition which involves both need for proper datasets, machine learning and deep learning models [5]. A considerable segment of the populace in Bangladesh utilizes Bangladeshi Sign Language (BdSL) for routine communication [6]. A digital communication tool that closes the gap between signers and non-signers is therefore necessary. But even though computers are getting better at identifying hands in pictures [7, 8], there isn't much focus on real-time, speedy recognition of individual sign letters, especially when it comes to Bangladeshi Sign Language. This difficulty is made much more difficult by the dearth of relevant datasets designed specifically to aid in this undertaking.

1.3 Objectives

The primary objective of this research is to develop a comprehensive dataset of Bangla Sign Language (BdSL) gestures. Recognizing the importance of data for training machine learning and deep learning models, a dataset of 12,000 photos was curated, capturing the subtle motions and expressions of BdSL. This dataset serves as the foundation for future model development and evaluation. The next goal is to deploy machine learning and deep learning models to detect BdSL gestures accurately and efficiently. Using advanced algorithms and neural network architectures, the research aims to harness artificial intelligence to decode the complexities of sign language communication. After model implementation and performance evaluation, the focus shifts to optimizing and improving the models for better accuracy and robustness. This involves exploring cutting-edge techniques and fine-tuning model param-

eters. The ultimate goal is to apply these findings to real-world situations, facilitating easy communication between signers and non-signers and promoting accessibility and inclusivity for people with hearing loss.

1.4 Social Impact

This study aims to improve the quality of life for people with hearing impairments by developing accurate sign language identification systems particular to Bangla Sign Language (BdSL). The study's goal is to develop more understanding and acceptance in society by allowing for smooth interactions between signers and nonsigners. The study addresses common stigmas and misconceptions about hearing impairments by increasing communication between those who use sign language and those who don't, resulting in a more compassionate and welcoming community. Furthermore, the implications of this research go beyond interpersonal interactions and include educational, healthcare, and employment settings. Improved communication accessibility is expected to result in more inclusive learning environments, better healthcare outcomes, and increased integration of people with hearing impairments into the workforce, increasing chances for economic inclusion. Finally, this study advocates for the rights and dignity of persons with hearing impairments, aiming to enable their full involvement in social, economic, and cultural arenas while also contributing to the creation of a more equitable and inclusive society.

1.5 Issues Encountered

During this research, several significant obstacles were encountered. The lengthy data collection process required traveling to various schools in Bangladesh to gather pictures from deaf and mute autistic students. Building trust and meaningful relationships with these students was challenging and time-consuming. Substantial travel and logistical coordination added complexity. After obtaining the dataset, implementing various machine learning and deep learning algorithms proved difficult. These algorithms generated numerous results, such as graphs and matrices, making it tough to choose the best approaches and analyze the outcomes. Balancing practical real-world application issues with the technical complexities of algorithm selection required careful navigation and iterative refinement.

1.6 Organization of the Thesis

The book begins with a thorough examination of sign language as a natural form of communication, emphasizing its intrinsic value and importance for smooth communication, especially for those who are deaf or hard of hearing. The necessity of effective recognition systems in sign language technology is highlighted, demonstrating how they facilitate better communication between signers and non-signers in various real-world situations. In the next chapter, a comprehensive review of existing research in the field is provided, analyzing the methods used and identifying limitations in previous sign language recognition studies. Following this, the methodically developed approach and detailed implementation of the proposed algorithm for sign language recognition are described. This involves applying state-of-the-art machine learning and deep learning algorithms, as well as creating a broad and varied dataset with diverse sign language motions and expressions. The domain of result analysis is addressed with a careful examination of experimental results, offering insightful comparisons with current recognition methodologies. The final section summarizes the key findings, explaining their significance for the development of sign language recognition technology and suggesting future directions for this important field of study. This research work is publicly available at GitHub repository [9].

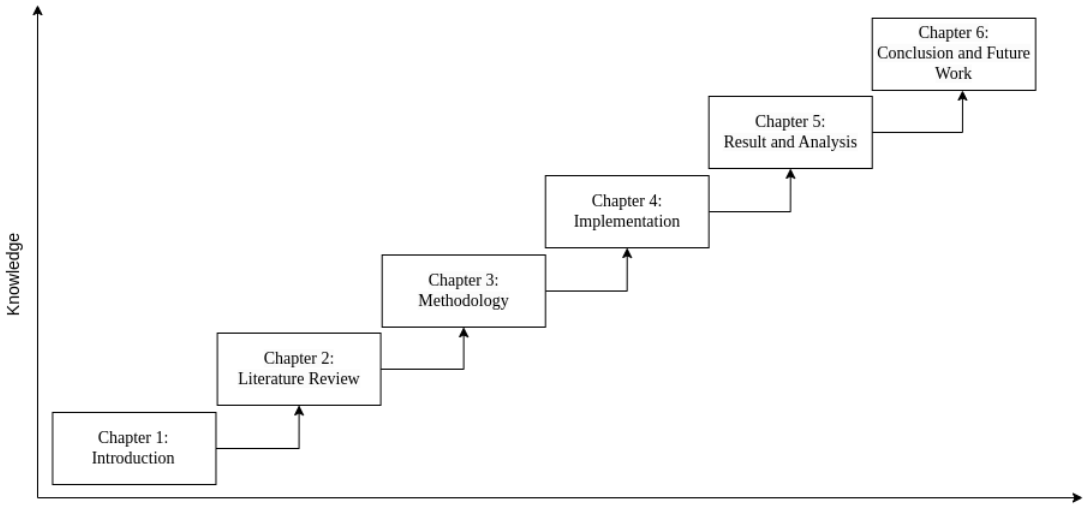


Figure 1.2: Organization of the thesis' chapters.

CHAPTER 2
Literature Review

2.1 Overview

The literature on Bangladeshi Sign Language (BdSL) recognition highlights key limitations. Various deep learning architectures focus on real-time interpretation and background augmentation. However, issues like misclassifications, biases from low volunteer engagement, and data size limits persist. Studies often describe Bengali consonants using diverse participant data but are limited in scope. Depth information and neural networks are employed, yet real-world applicability is constrained by controlled data collection. BdSL word recognition algorithms show good accuracy but suffer from poor model selection and data diversity. Larger, more varied datasets and ensemble approaches are needed to improve BdSL systems.

Title: BDSL 49: A comprehensive dataset of Bangla sign language [10].
The paper introduces a dataset called BDSL49, designed for Bangla Sign Language (BDSL), which is used by hearing-impaired and nonverbal individuals communicating in Bengali. The dataset is freely available to researchers and aims to support the development of automated systems using machine learning, computer vision, and deep learning techniques. The paper also details the application of two models on this dataset: one for detecting the presence of signs and another for identifying and classifying specific signs.
Methodology: The dataset consists of 29,490 images annotated with 49 labels, encompassing 37 alphabets from the Bangla language, 10 numeric digits, and 2 special characters. These images were meticulously collected from 14 distinct adult participants to ensure a diverse and comprehensive representation. For image detection tasks, the YOLOv4 model was employed, leveraging its robust capabilities in object detection. For image recognition, a variety of models were utilized, including Xception, InceptionV3, InceptionResNetV2, ResNet50V2, and DenseNet, each chosen for their performance.

Results: Detection and recognition accuracy of different models for BDSL 49 are summarized in Table 2.1, showcasing their performance in tasks related to object detection and image recognition.

Table 2.1: Detection and recognition results for BDSL9 [10].

Task	**Models**	**Accuracy (%)**
Detection	YOLOv4	99.4
Recognition	Xception	93.0
	InceptionV3	87.0
	InceptionResNetV2	91.0
	ResNet50V2	86.0
	DenseNet	91.0

Limitations:

- **Limited Diversity in Data Collection:** The dataset was collected from only 14 individuals, which may not fully represent the diversity of hand gestures and signing styles present in the Bangla sign language community. This limited diversity could affect the generalizability of the models trained on this dataset.
- **Absence of Custom Models:** The research solely relies on pre-trained deep learning models for sign language recognition. While these models showed promising results, the absence of custom models specifically tailored to the nuances of Bangla sign language might limit the accuracy and effectiveness of the recognition systems.
- **No Ensemble Modeling:** Ensemble modeling, which combines predictions from multiple models to improve accuracy, was not explored in this research. Employing ensemble methods could potentially enhance the overall performance of the sign language recognition systems.
- **Limited Accuracy of Models:** Despite achieving reasonable accuracy, none of the models reached top-notch performance on the dataset. This suggests that there is still room for improvement in the recognition accuracy of Bangla sign language signs, possibly through the development of more sophisticated models or the augmentation of the dataset.
- **Challenges in Real-world Deployment:** The research does not address the challenges associated with deploying sign language recognition systems in real-world scenarios.

Title: Shongket: A Comprehensive and Multipurpose Dataset for Bangla Sign Language Detection [11].

The paper presents a new Bangla Sign Language (BdSL) dataset for improving communication between speech/hearing impaired individuals and others. It includes images of BdSL letters and digits collected from volunteers, with variations to enhance training accuracy. The study addresses the scarcity of open-access BdSL datasets in Bangladesh.

Methodology: The methodology involved collecting 5,820 images depicting gestures for 10 Bangla digits and 36 Bangla letters, totaling 46 classes. A Convolutional Neural Network (CNN) with four convolutional layers, ReLU activation, max-pooling, and dropout layers was used for recognition. The network's output layer consisted of 36 units for letters and 10 for digits, with integrated classifiers including K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Random Forest, and Decision Trees to enhance classification performance.

Results: Classification accuracies of different models for letters and digits are summarized in Table 2.2 and Table 2.3 respectably, showcasing their performance in image classification.

Table 2.2: Classification results of letters for Shongket [11].

Models	**Dataset**	**Accuracy (%)**
KNN	Letter	91.0
Random Forest	Letter	91.2
SVM	Letter	85.2
Decision Tree	Letter	91.3

Table 2.3: Classification results of digits for Shongket [11].

Models	**Dataset**	**Accuracy (%)**
CNN	Digit	95.0
KNN	Digit	95.0
Random Forest	Digit	93.8
SVM	Digit	87.8
Decision Tree	Digit	93.5

Limitations:

- **Limited Quantity:** Despite efforts to collect a substantial dataset, the total number of images (5820) may still be insufficient to fully capture the diversity and complexity of sign language gestures. A larger dataset size could potentially lead to improved model performance.

- **Limited Evaluation Metrics:** The research primarily focuses on accuracy as the evaluation metric, but other metrics such as precision, recall, and F1-score could provide a more comprehensive assessment of model performance.
- **Model Performance:** While the reported accuracies are commendable, none of the applied models achieved top-notch accuracy. This suggests that there may be room for improvement in the models' performance, potentially due to the complexity and variability of sign language gestures.
- **No Ensemble Methods:** The researchers did not utilize ensemble methods, which could potentially enhance model performance by combining predictions from multiple models. The absence of ensemble methods may have limited the models' ability to achieve higher accuracies.

Title: BdSL36: A Dataset for Bangladeshi Sign Letters Recognition [12].
The paper introduces BdSL36, a dataset designed for real-time Bangladeshi Sign Language (BdSL) interpretation in uncontrolled environments. BdSL36 includes background augmentation and annotations with bounding boxes to aid object detection algorithms. Baseline performance experiments and user-level beta testing show its real-world applicability. BdSL36 aims to advance research in sign letter classification, offering both dataset and pre-trained models for further exploration. BdSL36 represents a significant step towards enhancing accessibility and inclusivity through real-time Bangladeshi Sign Language interpretation.
Methodology: The methodology began with the collection of 1,200 images by 10 volunteers, capturing Bangladeshi Sign Language (BdSL) gestures in natural settings to initiate BdSL letter recognition. Additionally, BdSlImset contributed 1,700 images distributed across 10 classes, resulting in a combined dataset of 2,712 images across 36 classes. Initial attempts at enhancing the dataset through traditional data augmentation methods yielded suboptimal performance in real-world applications. Subsequently, BdSL36v1 was enriched to encompass a more extensive collection of approximately 26,713 images, ensuring an average of 700 images per class. The subsequent iteration, BdSL36v2, further augmented the dataset by introducing 473,662 images using advanced background augmentation techniques. In terms of modeling, the study adopted a diverse array of deep learning architectures, including ResNet34, ResNet50, VGGNet19, DenseNet169, DenseNet201, AlexNet, and SqueezeNet, leveraging their respective strengths as foundational models.

Results: Classification accuracy of different models for BdSL36 are summarized in Table 2.4, showcasing their performance in tasks related to image classification.

Table 2.4: Classification results for BdSL36 [12].

Models	**Accuracy (%)**
ResNet34	98.17
ResNet50	98.71
VGG19 with BN	99.10
DenseNet169	98.55
DenseNet201	98.56
AlexNet	83.10
SqueezeNet	41.50

Limitations:

- **Limited Real-World Data:** The initial dataset size of 1200 images, with 25 to 35 images per class, is relatively small for training deep learning models, especially for a multi-class classification task with 36 classes. Although data augmentation techniques are applied to increase the dataset size, it may still not be sufficient to capture the full diversity and variability of sign language gestures. Augmentation can introduce synthetic variations, but it may not capture the diversity of real-world scenarios accurately.

- **Dataset Collection Process:** While efforts were made to collect images in natural environments with variation in subjects and backgrounds, relying on 10 volunteers may introduce biases or inconsistencies in the dataset.

- **Misclassification:** The research showed that class 4,6, class 14,24 and class 29,30 are mostly misclassified by the classifiers. class 24, and class 14 are incredibly similar and class 24 has got the worst performance with less than 55% confidence rate and less than 40% accuracy with multiple tries.

- **Absence of Ensemble Models:** The absence of ensemble models in the study may limit the exploration of potential performane improvements by leveraging the strengths of different models.

- **Limited Model Exploration:** The study primarily focuses on using pre-trained deep learning models without exploring the potential benefits of custom model architectures.

Title: KU-BdSL: An open dataset for Bengali sign language recognition [13].
This research paper focuses on the development of the KU-BdSL dataset, a comprehensive collection of images representing Bengali Sign Language (BdSL) consonants. Given the complexity and extensive vocabulary of BdSL, understanding and using this sign language poses significant challenges.
Methodology: The dataset consists of 30 classes representing 38 consonants ('banjonborno') of the Bengali alphabet. Each class comprises 50 images, resulting in a total of 1,500 images collected. The images were sourced from 39 participants, including 30 males and 9 females, aged between 21 and 38 years, from various regions of Bangladesh.
Results: The Multi-scale Sign Language Dataset (MSLD) comprises original images captured at various scales to capture diverse perspectives. In contrast, the Uni-scale Sign Language Dataset (USLD) standardizes images to 512×512 pixels, ensuring uniformity for analysis and comparison. The Annotated Multi-scale Sign Language Dataset (AMSLD) enhances MSLD with annotations in YOLO DarkNet format, facilitating object detection tasks with detailed bounding box information.
Limitations:

- **Amount of Data:** The dataset has only 1500 images for 30 classes. This is not enough for training modern machine learning and deep learning models for using in real time.
- **Scope of Data:** The dataset focuses only on the consonants of the Bengali alphabet, leaving out vowels and word-level signs. This limits its application to a subset of BdSL communication.
- **Participant Demographics:** The participants are all aged between 21 and 38, with no representation from children or senior citizens, which might affect the generalizability of the dataset to all age groups.
- **Participant Demographics:** The participants are all aged between 21 and 38, with no representation from children or senior citizens, which might affect the generalizability of the dataset to all age groups.

Title: BdSL47: A complete depth-based Bangla sign alphabet and digit dataset [14].
The paper introduces BdSL47, a comprehensive depth-based dataset for Bangla Sign Language (BdSL) recognition, addressing the scarcity of such resources. BdSL47 includes 47

one-handed static signs, comprising letters and digits. The dataset consists of jpg images and CSV files with normalized 3D coordinates of hand key points.

Methodology: RGB image samples were collected from 10 signers to accurately capture hand gestures, with over 150 samples initially gathered per sign and a final selection of 100 samples per sign chosen for accuracy. These images underwent preprocessing, including resizing to a standard size, min-max normalization, and depth value standardization, ensuring dataset uniformity. Hand key-points were detected using the MediaPipe framework to precisely identify gestures, with faulty samples due to issues like poor lighting or blur manually removed to maintain dataset quality. The Bangla sign alphabet and sign digit datasets were merged to create the BdSL47 dataset, adjusting labels for accurate classification. Traditional machine learning models (KNN, SVM, LR, RFC) and ensemble methods (Hist-GBC, Light-GBM) established a classification baseline, with parameters optimized through empirical research. Additionally, an Artificial Neural Network (ANN) model with an input layer of 63 nodes, four hidden layers (including dropout), and an output layer was employed.

Results: Classification accuracy of different models for BdSL47 are summarized in Table 2.5, showcasing their performance in tasks related to image classification.

Table 2.5: Classification results for BdSL47 [14].

Models	**Accuracy (%)**
KNN	98.49
SVM	96.55
LR (Logistic Regression)	86.31
RFC (Random Forest Classifier)	98.53
HistGBC (Histogram-Based GB Classifier)	98.81
Light-GBM	98.84

Limitations:

- **Missclassification:** ANN model has made few misclassifications in cases of some particular signs (e.g. Sign 21, 26, 32, 38, 43, 44, 45), because some of the digit signs are almost identical to some of the letter signs.
- **Controlled Environment:** The proposed dataset limits natural variations in the data due to controlled settings while collecting Bangla sign images. This will result degradation in accuracy wile performed in real-world environment. Also this dataset does not include dynamic background which is crucial for real-time sign language classification.

Title: BdSLW-11: Dataset of Bangladeshi sign language words for recognizing 11 daily useful BdSL words [15].

The BdSLW-11 dataset represents a significant step forward in supporting the communication needs of the D&D community in Bangladesh. Its comprehensive coverage of common sign words, coupled with rigorous data collection and analysis methods, makes it a valuable asset for researchers seeking to develop innovative solutions for BdSL recognition.

Methodology: This research focused on developing the Bangladeshi Sign Language Words (BdSLW) dataset, aimed at recognizing 11 commonly used sign words: 'Bad', 'Beautiful', 'Friend', 'Good', 'House', 'Me', 'My', 'Request', 'Skin', 'Urine', and 'You'. These hand signs were manually captured from real volunteer signers with their consent. The dataset was evaluated using deep learning models, specifically employing transfer learning (TL) techniques. Models such as VGG16, VGG19, InceptionV3, ResNet-50, and AlexNet were trained on the BdSLW dataset to assess their performance in recognizing the 11 BdSL sign words.

Results: Classification accuracy of different models for BdSLW-11 are summarized in Table 2.6, showcasing their performance in tasks related to image classification.

Table 2.6: Classification results for BdSLW-11 [15].

Models	**Accuracy (%)**
VGG16	99.92
VGG19	99.58
InceptionV3	98.70
ResNet50	68.66
AlexNet	97.86

Limitations:

- **Limited Data:** The dataset is a collection of sign images consisting of 1105 images. The range of total images of each class is 77 to 132. This is very low amount for modern classifer for robust output.
- **Limited Diversity of Data:** The background of the images is kept as same for better recognition accuracy. So, there are not enough variations and lighting.
- **Limited Model Selection:** The research have selected only pre-trained model inseat of using any custom or ensemble method for the task.

Title: Computer vision-based six layered ConvNeural network to recognize sign language for both numeral and alphabet signs[16].

The research delves into Sign Language Recognition (SLR) systems, addressing critical limitations in existing studies. Recognizing the challenges faced by individuals with speech and/or hearing impairments, the study focuses on developing an effective SLR system to bridge communication gaps. By creating two datasets, BdSL_OPSA22_STATIC1 and BdSL_OPSA22_STATIC2, comprising images of Bangla characters and numerals with diverse backgrounds, the research aims to overcome the scarcity of large, varied datasets.

Methodology: This research focused on two datasets for Bangladeshi Sign Language (BdSL) recognition: BdSL_OPSA22_STATIC1 and BdSL_OPSA22_STATIC2. The first one comprises 24,615 images categorized into Numerals (2,695 images), Vowels (6,146 images), and Consonants (15,774 images), contributed by 7 individuals (5 males, 2 females). BdSL OPSA22 STATIC2 includes 8,437 images categorized as Indoor Solid (2,823 images), Outdoor Ambiguous (1,925 images), and Indoor Cluttered (3,689 images), contributed by 7 individuals (4 males, 3 females). Image pre-processing involved grayscale conversion (0 to 255 range), normalization to [0, 1], and conversion to NumPy arrays with labels transformed into one-hot encoded vectors. The study utilized Convolutional Neural Networks (CNNs) for feature extraction, employing a model named "ConvNeural" comprising six layers: four convolutional layers and two fully connected layers.

Results: Classification accuracy of different models are summarized in Table 2.7, showcasing their performance in tasks related to image classification.

Table 2.7: Classification results for BdSL_OPSA [16].

Models	**Dataset-1 (%)**	**Dataset-2 (%)**
VGG16	94.88	94.88
VGG19	90.44	90.44
DenseNet-121	62.44	62.44
InceptionV3	66.72	66.72

Limitations:

- **Homogeneity of Dataset Contributors:** The dataset contributors may not fully represent the variation of individuals who use sign language, potentially leading to biases in the dataset because data was collected only from 7 persons.

- **Misclassification:** The system provided low accuracy for certain words because of equivalent postures [16].
- **Lack of Diversity:** 'BdSL_OPSA22_STATIC1' holds significant amount of data but all most of them are in solid backgrounds. 'BdSL_OPSA22_STATIC2' holds lesser amount of data in the ambiguous background. This causes the models trained with the dataset will not perform well in real-life scenarios because real life images will have diverse backgrounds.

2.2 Summary

The literature review comprehensively evaluates existing research on Bangla Sign Language Recognition, analyzing methodologies employed in previous studies, highlighting their strengths and limitations. This review establishes a solid understanding of the current research landscape in Bengali Sign Language Recognition. It identifies key themes, research findings, and gaps in the field that this thesis aims to address, laying the groundwork for subsequent chapters.

CHAPTER 3
Methodology

3.1 Overview

The methodology for researching BdSL recognition involved gathering data from diverse sources across Bangladesh. This included input from sign language interpreters, special education schools for impaired students, and experienced educators to ensure the inclusivity and accuracy of the dataset. Advanced techniques were applied throughout the process, including rigorous data collection, preprocessing, and model training. Systematic annotation, careful selection of model architectures, fine-tuning of hyperparameters, and thorough evaluation metrics were utilized to enhance the effectiveness and applicability of BdSL recognition algorithms. The collaborative effort in data gathering from varied sources and the meticulous application of advanced techniques underscore the robustness of the proposed methodology.

3.2 Data Collection and Preprocessing

A dataset is being created through the hand gathering of pictures from various sites around Bangladesh. This dataset contains 11,864 photos divided into 38 classes, each representing a different symbol of the BdSL. The gathering procedure seeks to create a comprehensive representation of BdSL symbols to aid in research and development in sign language recognition and related domains. This dataset will be a valuable resource for improving the accuracy and efficiency of sign language recognition systems.

3.2.1 Data Sources

To ensure diversity and comprehensiveness, data on BdSL recognition was gathered from multiple sources. These sources include people who use sign language as their primary form of communication and special education programs for kids with autism and other disabilities

including deaf and mute. To guarantee authenticity and accuracy, professionals and educators collaborated on the data collection process.

The primary data source was Proyash in Rangpur [17], a school for children with special needs, including those who are mute or deaf. Data was collected from both male and female students with the assistance of the school's coordinator, ensuring accurate sign language performance. Similarly, data was gathered from students at Mymensingh Welfare School [18] and Anando Mela in Mymensingh, with educators ensuring the authenticity of the signs. These institutions provided significant and diverse contributions to the dataset. Additional data was collected at SAND Autism School in Gazipur [19], where a supervising teacher ensured the accuracy of the signs. Other schools for children with disabilities also contributed, broadening the range of sign language data. Furthermore, data from deaf and mute adults, personally known to the me who are experienced in sign language, added valuable perspectives from adult users. This combination of sources, covering different age groups, genders, and usage contexts, ensures the dataset's representativeness and enhances the BdSL recognition models' generalizability and applicability.

3.2.2 Data Collection Techniques

The data collection for this research involved capturing images using smartphones, aided by volunteers. The primary tool for taking photos was the Open Camera app, which was configured to capture raw images rather than processed ones. This approach ensured the retention of unaltered visual data.

The images were taken in various environments to capture a wide range of lighting conditions and backgrounds. Locations included classrooms, playgrounds, canteens, corridors, living rooms, outdoor roads, and more. The lighting conditions ranged from harsh natural sunlight to dim room light, near darkness, and both day and night settings. This diversity in environmental settings aimed to enhance the dataset's robustness and real-world applicability. A variety of smartphones were used for image capture, including the Redmi Note 12 Pro, Realme 11 Pro, Samsung M32, Oppo Reno 7, Realme 8, Poco M2 Pro, Infinix Hot 10, and Redmi Note 8. Despite differences in camera resolution among these devices, the Open Camera app was uniformly set to capture 12 MP images with an aspect ratio of 1:1. Consistent camera settings across all devices ensured uniformity in the collected data.No stands were used for the smartphones during image capture, allowing the photos to reflect natural,

uncontrolled settings. This approach contributed to the dataset's uniqueness by incorporating chaotic backgrounds rather than typical, controlled environments, enhancing the dataset's realism and applicability to real-life scenarios.

3.2.3 Preprocessing Steps

The preprocessing steps involved in preparing the raw data for model training and analysis. The flowchart shown in Figure 3.3 illustrates a process for preparing raw images for inclusion in a dataset. It begins with the acquisition of raw data, specifically raw images. These images undergo an initial assessment to determine their usability. If an image is seemed unusable, it is deleted to ensure only quality data is processed further. If it is usable, the image enters the raw data processing stage. This stage involves several critical steps: first, standardizing the aspect ratio to 1:1, ensuring uniformity across all images; and second, resizing the image to a resolution of 224x224 pixels, which is a common size used in various machine learning and image processing tasks. By converting the images to a consistent aspect ratio and resolution, the process ensures that the dataset is homogeneous, which can improve the performance of algorithms trained on this data. After processing, the standardized images are stored in the dataset, ready for further use or analysis, such as training machine learning models or conducting further image analysis. This meticulous preprocessing ensures that the dataset is both high-quality and consistent, providing a solid foundation for any subsequent computational tasks.

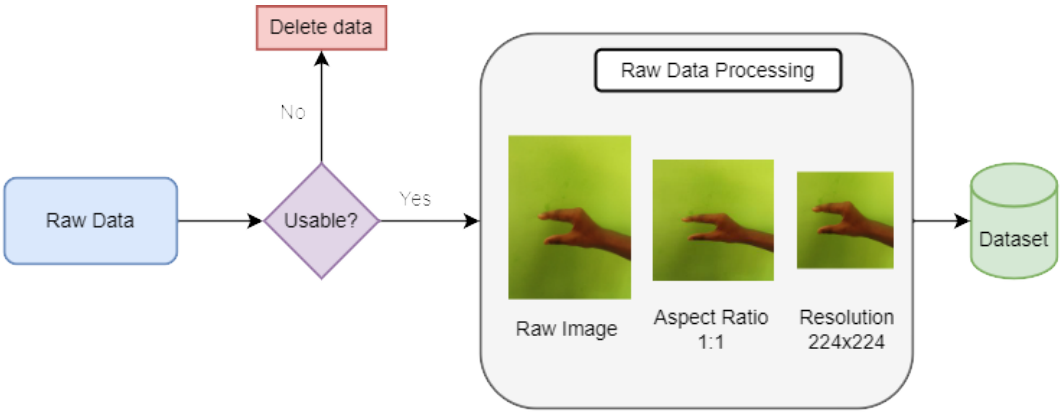


Figure 3.3: Raw data processing flowchart.

Image Resize and Rename: The collected images underwent resizing to a standard resolution of 224x224 pixels using the Lanczos interpolation method. Lanczos interpolation works by using a weighted average of nearby pixels to calculate the new pixel value, resulting in

smoother and higher quality resized images compared to other interpolation methods. This resizing ensured uniformity in image dimensions across the dataset, facilitating efficient processing and model training. Additionally, each resized image was renamed with a unique identifier indicating its class, aiding in dataset organization and management.

The Lanczos kernel $L(x)$ is defined as:

$$L(x) = \begin{cases} \text{sinc}(x) \cdot \text{sinc}\left(\frac{x}{a}\right), & \text{if } -a \leq x \leq a \\ 0, & \text{otherwise} \end{cases}$$

where $\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$ is the sinc function, and a is the scale factor determining the size of the kernel. The interpolation formula using the Lanczos kernel is:

$$I_{\text{interp}}(x) = \sum_{n=-N}^{N} I(n) \cdot L(x - n)$$

where $I_{\text{interp}}(x)$ is the interpolated pixel intensity at position x, $I(n)$ is the intensity of the input pixel at position n, N is the number of neighboring pixels considered in the interpolation, and $L(x-n)$ is the Lanczos kernel evaluated at the distance between the output pixel position x and the neighboring input pixel position n [20].

Orientation Correction: Images containing Exif metadata with orientation tags were adjusted to ensure uniform orientation across all images. This correction process involved rotating images by 180, 270, or 90 degrees, depending on the orientation tag values (3, 6, or 8, respectively). This meticulous alignment was crucial to maintain consistency throughout the dataset, preventing any orientation discrepancies that might otherwise affect the accuracy and reliability of subsequent image processing and analysis tasks. By standardizing the orientation, the images were prepared optimally for use in various applications, enhancing overall model performance and ensuring reliable results in further analyses and visualizations.

Data Augmentation: Various data augmentation techniques were applied to increase dataset diversity and robustness. These techniques included random rotation (up to 20% rotation), random translation (up to 10% height and width translation), random zooming (up to 20% zoom), horizontal flipping, and random contrast adjustment (up to 20%). In addition, to further simulate real-world variations and improve model generalizability, techniques like random shearing, color jittering, and Gaussian blur were also employed. By incorporating these

diverse augmentation strategies, the model's ability to recognize signs under different conditions and reduce overfitting was significantly improved.

Random Rotation: Calculate the rotation matrix:

$$\text{Rotation Matrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

For each pixel in the original image, apply the rotation matrix to calculate the new coordinates:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \text{Rotation Matrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Random Translation: Determine the translation matrix:

$$\text{Translation Matrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} \Delta h & 0 \\ 0 & \Delta w \end{bmatrix}$$

Apply the translation matrix to each pixel in the original image to calculate the new coordinates:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \text{Translation Matrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Random Zooming: For each pixel in the original image, scale the coordinates by the random zoom factor:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \alpha \begin{bmatrix} x \\ y \end{bmatrix}$$

Horizontal Flipping: For each pixel in the original image, apply the flipping matrix to flip horizontally:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Image Normalization: Min-max scaling, also known as normalization, is a preprocessing technique used to transform data to a specified range, commonly [0, 1] or [-1, 1]. The images were normalized using the Min-Max scaling method, where pixel values were scaled to

a range of [0, 1] by dividing each pixel value by 255. This rescaling ensures that all features contribute equally to the model, improving the performance and convergence speed of machine learning algorithms, especially those sensitive to input data scale like neural networks. Normalization ensures consistency in pixel intensity across images, mitigates the impact of varying illumination conditions, and enhances model stability and performance. By providing uniform and standardized input data, min-max scaling enhances the effectiveness of tasks such as image classification and segmentation. Min-max scaling is a popular method used in data preprocessing to scale features to a specified range. The formula for min-max scaling is:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

To scale to a different range $[a, b]$, the formula is:

$$x' = a + \left(\frac{x - \min(x)}{\max(x) - \min(x)} \right) \cdot (b - a)$$

For images, pixel values are often scaled to [0, 1] using:

$$x' = \frac{x}{255}$$

where x is the original pixel value [21].

Train-Test Split: The dataset was split into training and validation sets using Random splitting shown in Figure: 3.4 [22]. By shuffling the data randomly and allocating a portion, often defined by a specified ratio, to the validation set, this method ensures that the resulting subsets represent the overall dataset's diversity and characteristics. Random splitting helps prevent any biases that may arise from the inherent order or structure of the dataset, allowing for more robust model training and evaluation. Additionally, it supports the generalization of the trained model to unseen data by exposing it to varied examples during training and validation. Overall, random splitting is a simple yet effective approach for partitioning datasets, facilitating reliable model development and assessment. The images are selected through a process of randomization and allocation based on the specified validation split ratio. Initially, the images are shuffled randomly to prevent any inherent order bias. Following shuffling, a portion of

the dataset corresponding to the validation split ratio, typically set at 80% for training and 20% for validation, is set aside for validation. The remaining images are then allocated to the training set. It's important to note that this selection process occurs independently for each epoch during training, ensuring variability in the data seen by the model. Once assigned to either the training or validation set, the order of images remains consistent throughout the training process.

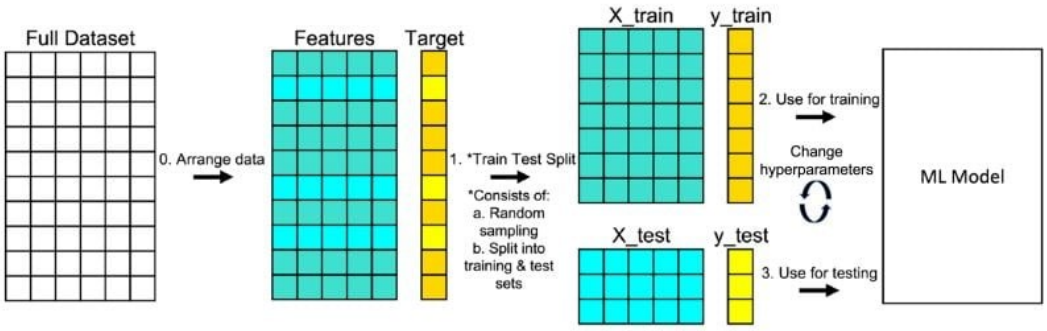


Figure 3.4: Random data splitting method [22].

3.2.4 Annotation Process

Annotation involves labeling data with descriptive metadata for efficient organization, retrieval, and analysis. In machine learning, it assigns meaningful labels or tags to data instances, crucial for supervised learning tasks. This iterative process requires careful consideration of domain-specific nuances and context, ensuring the accuracy and reliability of the dataset and enhancing model interpretability. Annotation methodologies vary, from simple categorical labels to complex hierarchical structures, depending on the data and task. Effective annotation is essential for unlocking the full potential of data-driven solutions to real-world problems.

The Bangla language, known for its rich script, includes "Sorborno" and "Banjonborno". "Sorborno" consists of 11 characters, while "Banjonborno" contains 39 characters, totaling 50 characters in Bangla. However, when represented as sign symbols, some characters combine into a single sign. Consequently, "Sorborno" is represented by 9 symbols and "Banjonborno" by 29 symbols, totaling 38 sign symbols. These 38 symbols serve as classes for the dataset in Table 3.8. Numerical notations were used to label the dataset, with each folder representing a symbol from the chart. The annotation process involves more than just character count-

ing. It includes visually representing these symbols to reflect their real-world appearance. This requires meticulous manual verification and cross-referencing to ensure each folder accurately represents the assigned symbols. Such rigorous validation minimizes mislabeling or misclassification, maintaining the dataset's integrity. It involves meticulously verifying each symbol against established standards and cross-referencing with authoritative sources to ensure accuracy. This systematic approach not only enhances the reliability of machine learning models trained on the dataset but also contributes significantly to scholarly understanding by highlighting the complexities inherent in Bangla script representation in digital contexts.

Table 3.8: Annotations of the symbols.

Serial	Character	Annotation	Serial	Character	Annotation
1	অ/অ	0	21	ট	20
2	আ	1	22	ঠ	21
3	ই/ঈ	2	23	ড	22
4	উ/ঊ	3	24	ঢ/ড়	23
5	ঋ/ঌ/ৄ/ৠ	4	25	ণ	24
6	এ	5	26	ত	25
7	ঐ	6	27	থ	26
8	ও	7	28	দ	27
9	ঔ	8	29	ধ	28
10	ক	9	30	প	29
11	খ/ক্ষ	10	31	ফ/ভ	30
12	গ	11	32	ব	31
13	ঘ	12	33	ম	32
14	ঙ	13	34	য/জ/্য	33
15	চ	14	35	র	34
16	ছ	15	36	ল	35
17	জ/য	16	37	শ	36
18	ঝ	17	38	ষ	37
19	ঞ	18			
20	ট	19			

3.2.5 Dataset Description

The dataset, encompassing 11,864 images across 38 classes shown in Figure:3.5, serves as a foundational resource for advancing computer vision research, particularly in the realm of BdSL. With an average of approximately 312 images per class, researchers have a foundation

for training and testing machine learning algorithms effectively. The predominant use of .jpg format ensures seamless integration and compatibility across diverse platforms and frameworks within the field of computer vision. The Figure 3.6 displays a bar chart representing the class distribution in a dataset. The x-axis shows the different classes, labeled from 0 to 37, while the y-axis indicates the number of images in each class. The colors of the bars transition smoothly from dark blue to yellow, providing a visual gradient effect. Each bar's

Figure 3.5: Dataset annotation.

height signifies the number of images in that particular class. It is evident that the dataset is relatively balanced, with the number of images per class ranging between approximately 270 and 350. However, slight variations exist, with some classes having slightly more images than others. The balanced nature of this dataset suggests that it is suitable for training machine learning models, reducing the likelihood of bias towards any particular class. The majority of these images are stored in the .jpg format, ensuring compatibility and ease of use across different platforms and frameworks in the field of computer vision. Class 35 stands out with the highest image count, totaling 347, accounting for about 3% of the dataset. Conversely, Class 32, with 284 images, represents approximately 2% of the dataset. This balanced distribution across classes supports comprehensive exploration across a spectrum of visual recognition tasks, ranging from image classification to object detection and pattern recognition. Moreover, the dataset is enriched by contributions from 20 female and 26 male participants, ensuring a balanced gender representation that enhances the dataset's inclusivity and applicability to real-world scenarios. This diversity not only mitigates potential biases but also broadens the dataset's utility in addressing varied perspectives and contexts. By providing such a diverse and meticulously structured dataset, researchers are empowered to push the boundaries of computer vision capabilities. Researchers can confidently innovate and refine algorithms, aiming for robust performance across diverse applications and domains. This dataset not only fosters technical advancements but also promotes ethical considerations, contributing to a more equitable development of artificial intelligence.

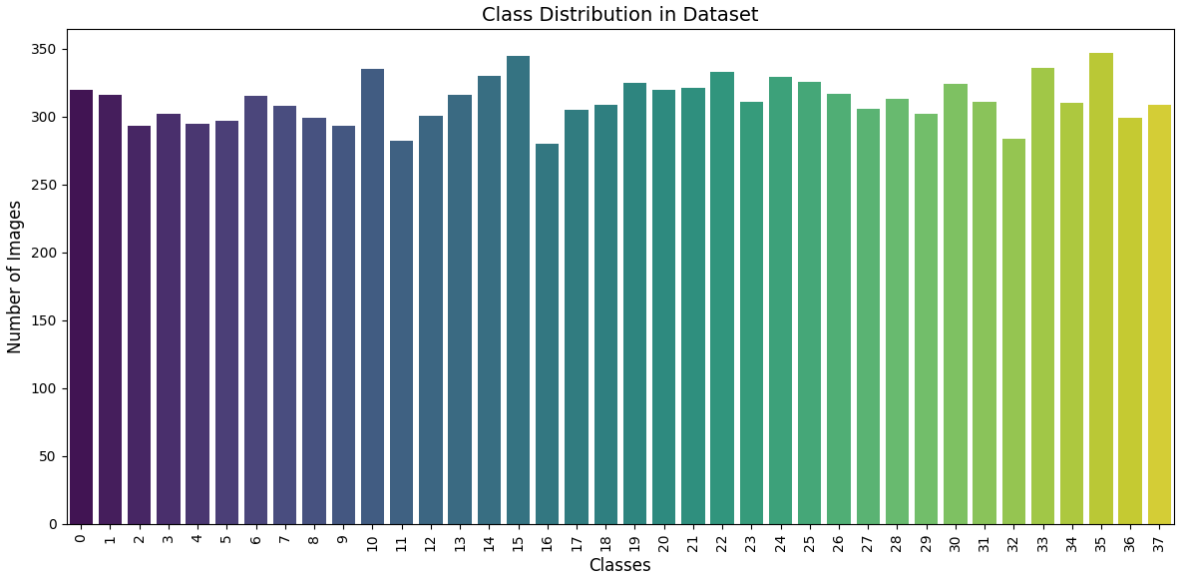


Figure 3.6: Class distribution in dataset.

3.3 Pre-trained Models for Comparative Analysis

3.3.1 Xception

Xception is a convolutional neural network known for its high accuracy and computational efficiency, making it ideal for real-time applications. It utilizes depthwise separable convolutions, dividing convolutions into depthwise and pointwise operations, significantly reducing parameters and computations shown in Figure 3.7. Additionally, Xception features residual connections to address issues like vanishing gradients, enhancing its effectiveness and stability in deep learning tasks [23].

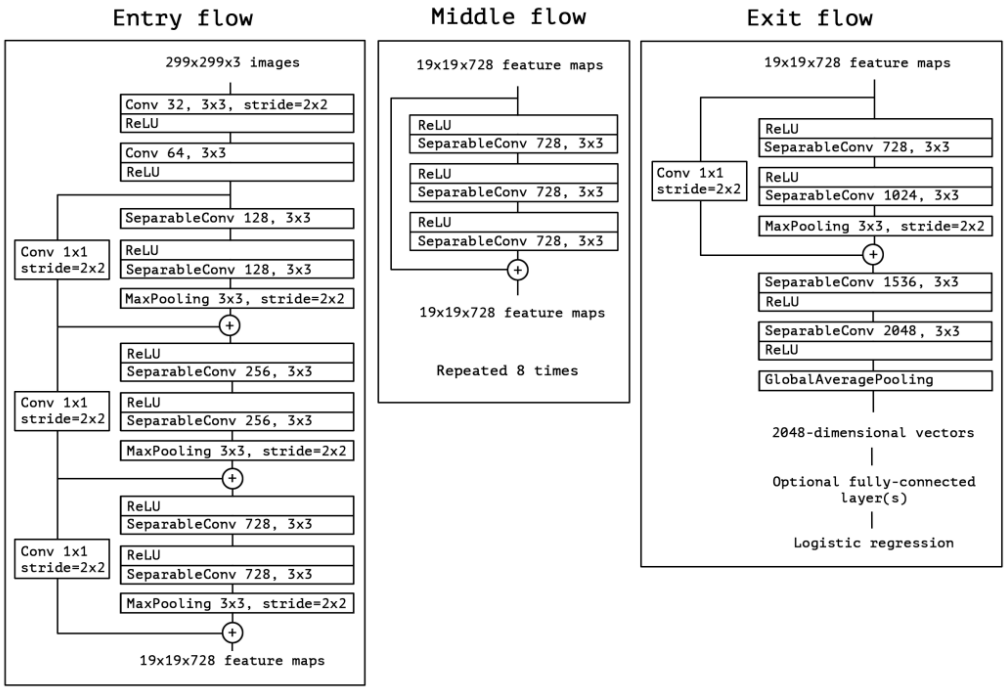


Figure 3.7: The Xception architecture [23].

3.3.2 Inception V3

Inception V3, a convolutional neural network architecture known for its strong performance in image classification, is distinguished by the usage of inception modules. These modules combine convolutional filters of varying widths into a single layer, allowing the network to successfully collect features at various scales. Inception V3's architecture includes stacked inception modules, each with simultaneous 1x1, 3x3, and 5x5 convolutional filters and a pooling layer shown in Figure 3.8. The diverse filter sizes enable the network to capture

fine details as well as broader patterns. This approach allows for simultaneous learning of multiple features, improving the network's capacity to extract and understand complex visual information quickly [24].

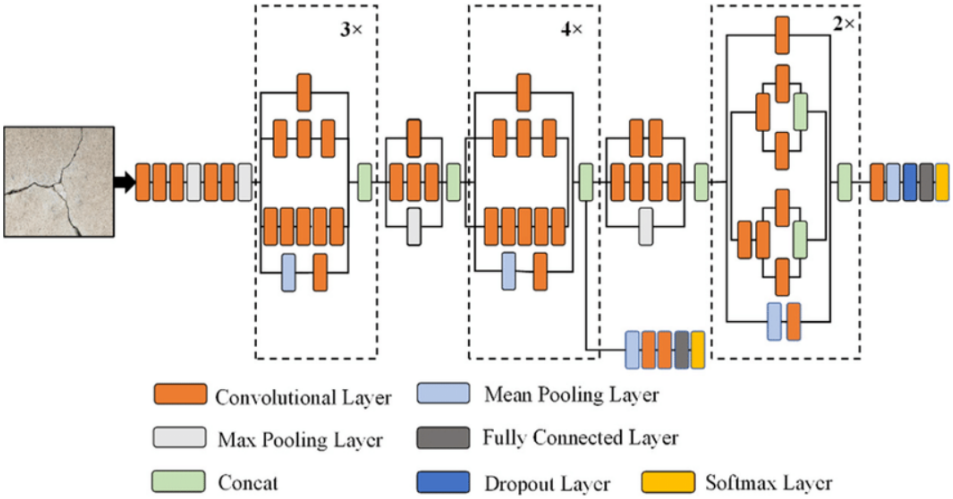


Figure 3.8: The InceptionV3 architecture [24].

3.3.3 ResNet-50

ResNet-50, part of the ResNet family developed by Microsoft Research, has 50 layers and uses residual blocks to achieve its depth. These residual blocks consist of convolutional layers along with shortcut connections that skip one or more layers, enabling the network to bypass certain transformations. This architectural innovation addresses the challenge of training very deep networks by facilitating the flow of gradients during backpropagation shown in Figure 3.9. As a result, ResNet-50 excels in tasks such as image classification and object detection, achieving state-of-the-art performance on benchmarks like ImageNet. Its success has influenced subsequent architectures, demonstrating the effectiveness of residual learning in advancing the capabilities of convolutional neural networks [25].

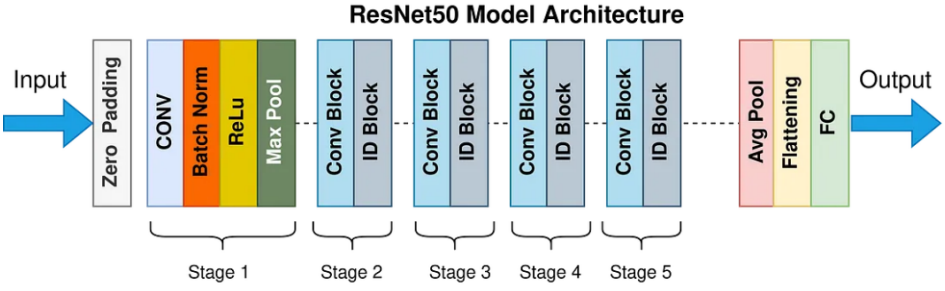


Figure 3.9: The ResNet50 architecture [25].

3.3.4 MobileNetV2

ResNet-50 uses shortcut connections, convolutional layers, and batch normalization to reduce vanishing gradients in deep networks. In contrast, MobileNetV2 was chosen for its lightweight and efficient design, which is critical in resource-constrained situations such as mobile and embedded devices. It has inverted residuals and linear bottlenecks, and it uses lightweight depthwise separable convolutions to reduce parameters shown in Figure 3.10 and computing costs while keeping excellent accuracy [26].

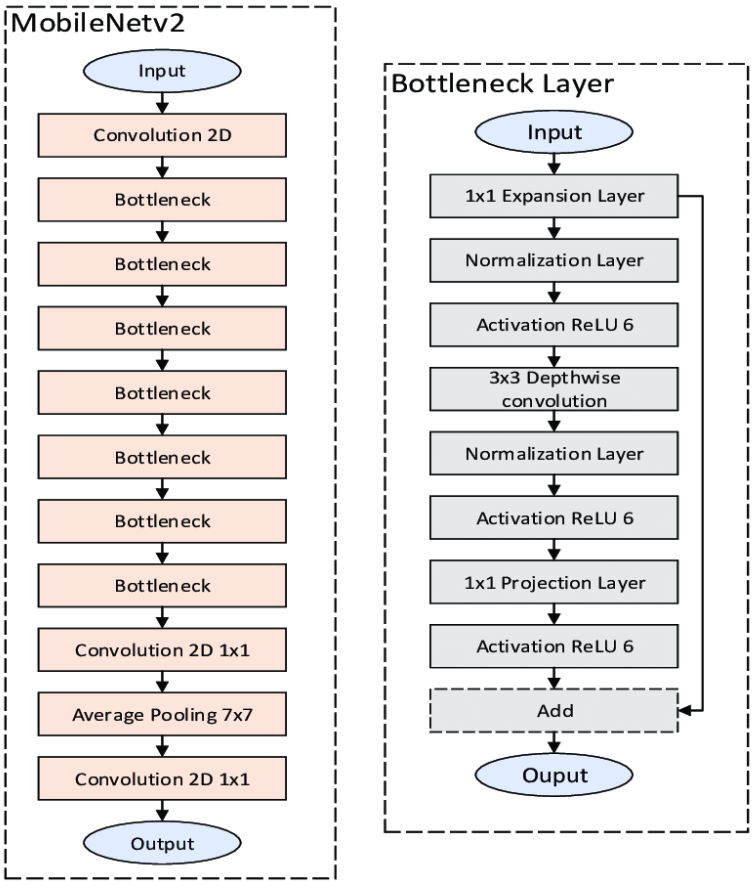


Figure 3.10: The MobileNetV2 architecture [26].

3.3.5 InceptionResNetV2

InceptionResNetV2 is chosen for its hybrid architecture, blending Inception modules with residual connections to enhance training stability and gradient flow in deep networks. This integration addresses challenges like vanishing gradients, leveraging the strengths of both approaches to achieve superior performance and efficiency. The architechture is shown in

Figure 3.11. The network's architecture allows for more effective feature extraction. This makes InceptionResNetV2 well-suited for managing complex features and variability in sign language data, crucial for applications demanding robust recognition capabilities on diverse, intricate datasets [27].

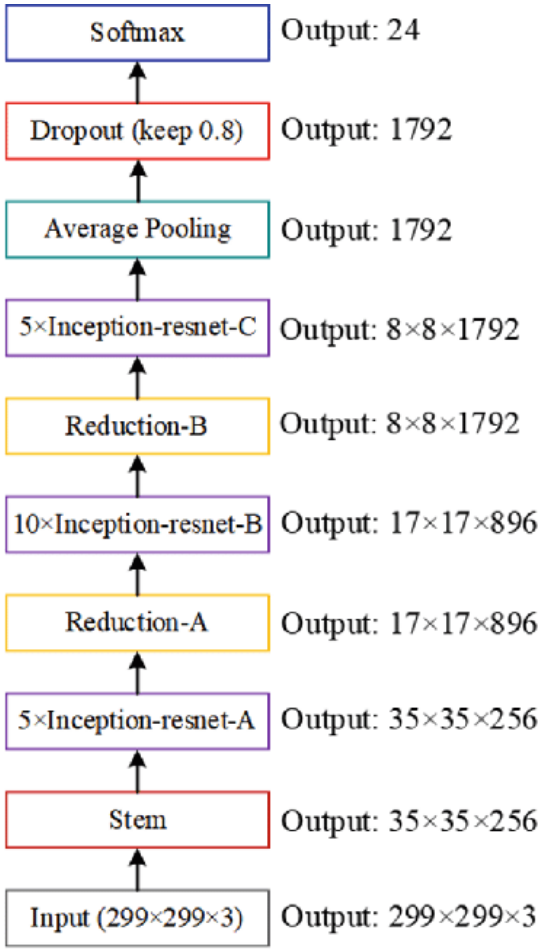


Figure 3.11: The InceptionResNetV2 architecture [27].

3.3.6 DenseNet121

DenseNet121 is selected for its unique dense connectivity pattern, where each layer is directly connected to every other layer in a feed-forward manner. This architecture promotes extensive feature reuse and facilitates robust gradient flow throughout the network, enabling efficient learning of complex representations while minimizing the number of parameters shown in Figure 3.12. The compact design of DenseNet121, coupled with its demonstrated high accuracy, renders it particularly suitable for environments constrained by computational resources yet demanding exceptional performance. Furthermore, the efficient layer connections

enhance information propagation and mitigate issues like vanishing gradients, ensuring stable and effective training even with deep networks. This makes DenseNet121 an ideal choice for applications requiring high performance with limited computational power, providing a balance of speed, accuracy, and resource efficiency [28].

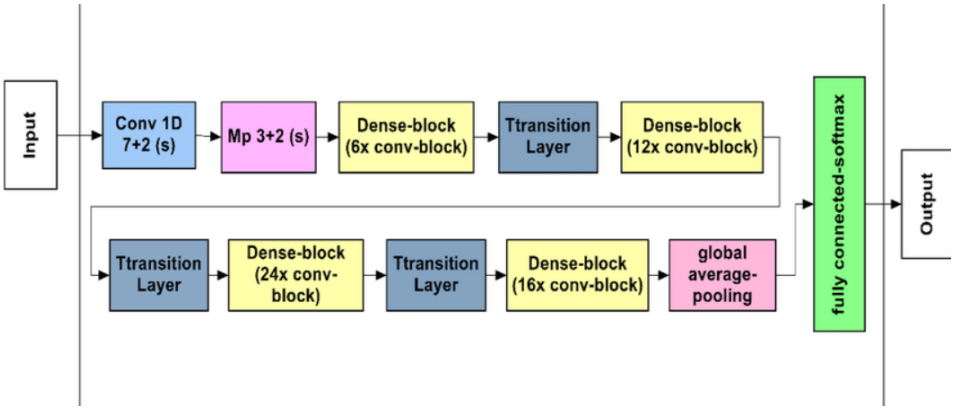


Figure 3.12: The DenseNet121 architecture [28].

3.3.7 DenseNet201

DenseNet201 is chosen for its deep architecture and dense connectivity, spanning 201 layers that enable comprehensive feature extraction across a wide range of scales and complexities. This depth is crucial for accurately discerning intricate nuances in sign language gestures. Despite its extensive depth, DenseNet201 maintains efficiency through effective parameterization shown in Figure 3.13, ensuring manageable computational overhead while delivering superior accuracy. This combination of depth and efficiency makes DenseNet201 well-suited for demanding image classification tasks that require detailed and high-fidelity analysis, particularly in scenarios such as sign language recognition [29].

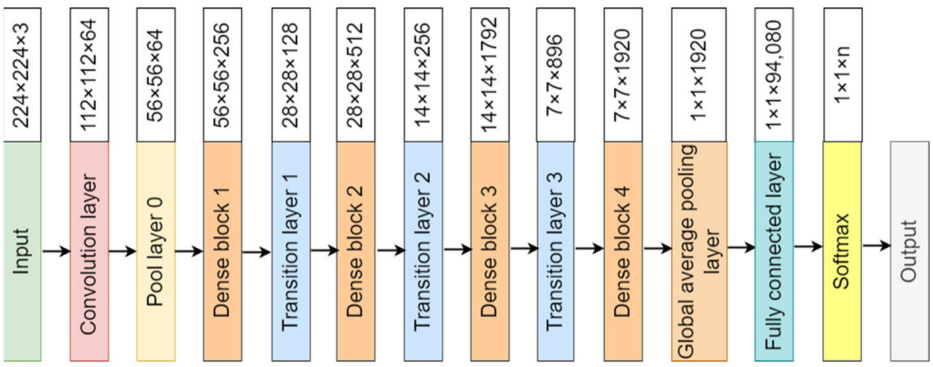


Figure 3.13: The DenseNet201 architecture [29].

3.3.8 VGG16

VGG16 is chosen for its widely recognized and deeply layered architecture, serving as a foundational model for numerous image classification tasks. Its straightforward design consists of multiple layers with 3x3 convolutional filters, which efficiently capture fine-grained details essential for distinguishing subtle nuances in sign language images. The architecture of VGG16 includes five convolutional blocks, each followed by max-pooling layers for spatial downsampling, culminating in three fully connected layers shown in Figure 3.14. Despite its depth of 16 layers, VGG16 remains computationally feasible for modern GPUs, ensuring practical deployment and robust performance across diverse backgrounds and applications requiring reliable image classification [30].

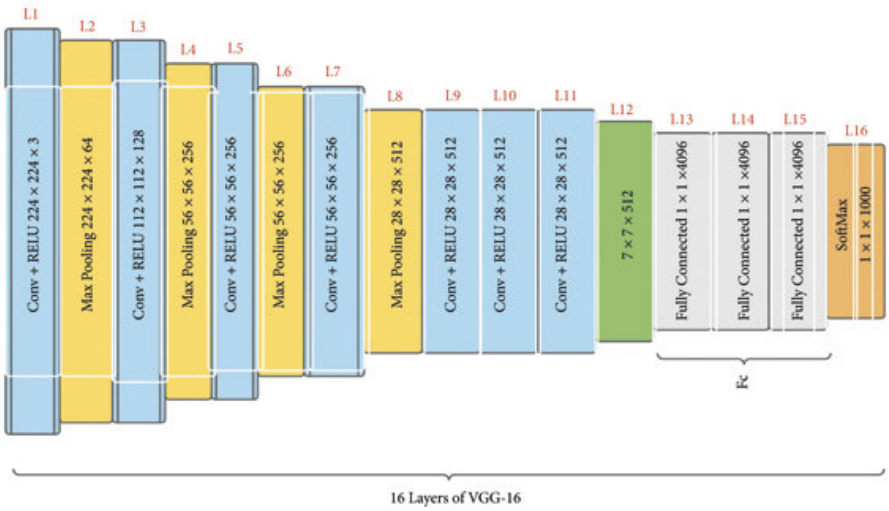


Figure 3.14: The VGG16 architecture [30].

3.4 Ensemble Techniques

Ensemble techniques combine multiple machine learning models to improve predictive performance over individual models. In this study, an ensemble model was proposed which is composed of three individual neural network models trained on the same dataset. The ensemble model aggregates predictions from these individual models to make a final prediction, leveraging the diversity of these models to enhance overall performance. The models were built and evaluated with motives to enhance the performance of recognition.

3.4.1 Ensemble Model 1: Xception Ensemble

This ensemble model leverages the pre-trained Xception architecture in a unique way to potentially improve Bangla sign language recognition performance. This ensemble model is created using a technique called multiple branch architecture with feature aggregation.

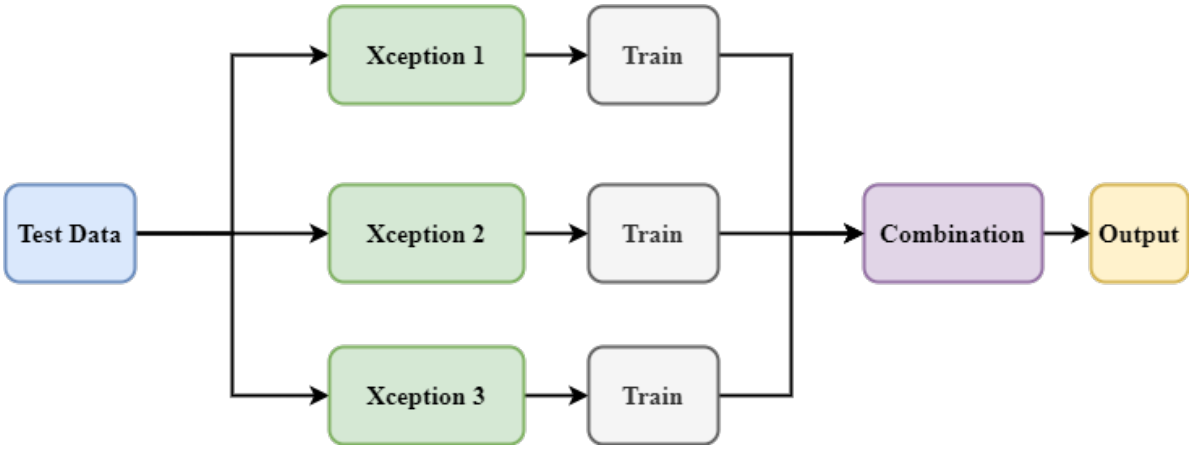


Figure 3.15: The Xception-Ensemble architecture.

Architecture

The ensemble model architecture shown in Figure 3.15 consists of three neural network models based on the Xception architecture, each with a similar structure but trained independently. The Xception architecture is chosen for its strong performance in image classification tasks and its ability to capture complex features from images effectively.

Base Model: Each individual model begins with a base Xception model pretrained on the ImageNet dataset. The base model serves as the feature extractor, capturing hierarchical representations of input images through a series of convolutional and pooling layers.

Output Layers: Following the base model, each individual model comprises additional layers for processing the extracted features and making predictions. These layers include:

1. Global Average Pooling: A global average pooling layer is added to the output of the base model to reduce the spatial dimensions of the feature maps while retaining important spatial information.

2. Dense Layers: Fully connected dense layers are used to further process the pooled features and extract high-level representations. These layers facilitate learning complex patterns and relationships within the data.

3. Dropout: Dropout layers are inserted between dense layers to prevent overfitting by randomly deactivating a fraction of neurons during training.

4. Output Layer: The final output layer consists of a dense layer with softmax activation, producing probabilities for each class in the classification task.

Ensemble Method

Once the individual models are trained, the ensemble architecture aggregates predictions from these models to make a final prediction. The ensemble architecture follows a simple averaging strategy, where predictions from each individual model are combined by taking the average of their probabilities for each class. Mathematically, the ensemble prediction $\hat{y}_{\text{ensemble}}$ for a given input x is computed as follows:

$$\hat{y}_{\text{ensemble}}(x) = \frac{1}{N} \sum_{i=1}^{N} \hat{y}_i(x)$$

where N is the number of individual models in the ensemble, and $\hat{y}_i(x)$ is the predicted probability distribution outputted by the i-th individual model for input x.

3.4.2 Ensemble Model 2: FusionNet

FusionNet represents a pioneering approach in the realm of machine learning, embodying the collaborative synergy of diverse neural architectures. With a name inspired by its fusion of multiple pre-trained models, FusionNet stands at the forefront of ensemble learning, harnessing the collective intelligence of renowned deep learning frameworks. Through the amalgamation of cutting-edge features from models like Xception, InceptionV3, ResNet50, and EfficientNetB0, FusionNet redefines the paradigm of predictive modeling, offering unprecedented accuracy and robustness. As a testament to the power of collaboration and integration, FusionNet symbolizes a new era in machine learning.

Architecture

FusionNet epitomizes a harmonious fusion of diverse deep learning architectures, meticulously crafted to achieve unparalleled performance in predictive tasks. The architecture of FusionNet shown in Figure 3.16 is characterized by its modular design, comprising several key components seamlessly integrated to maximize predictive accuracy and robustness.

1. Base Models Integration: At the core of FusionNet lies the integration of multiple pre-trained models, each renowned for its efficacy in feature extraction and representation learning. Specifically, FusionNet incorporates the convolutional bases of four distinguished models: Xception, InceptionV3, ResNet50, and EfficientNetB0. These models serve as the foundation for capturing intricate patterns and semantic features within the input data. By leveraging the strengths of each model, FusionNet aims to enhance the overall performance and robustness of the feature extraction process.

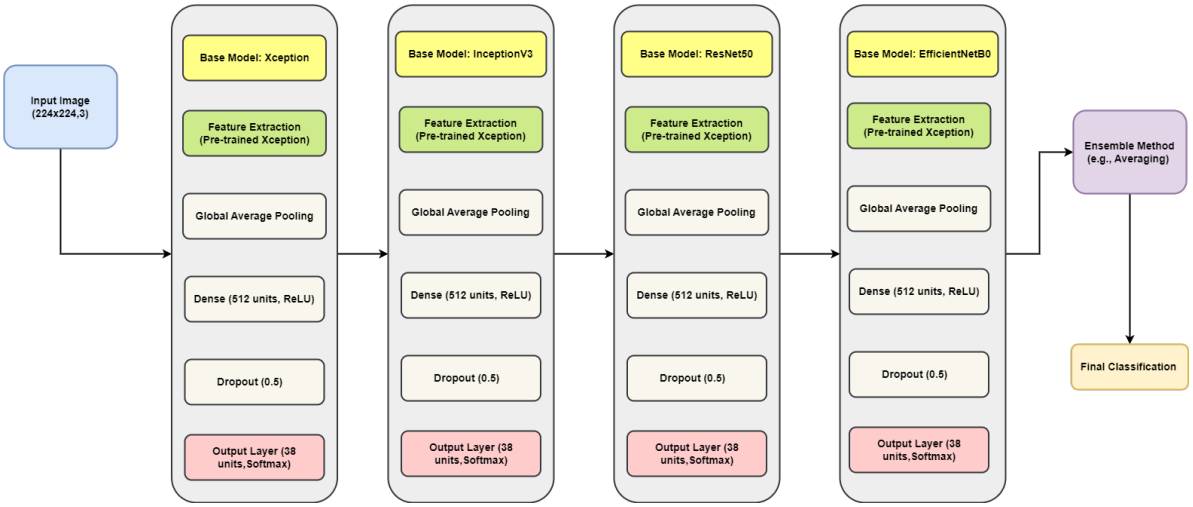


Figure 3.16: The FusionNet architecture.

2. Feature Fusion Layer: Following feature extraction by the base models, FusionNet incorporates a feature fusion layer to amalgamate the extracted features from each model. This fusion process enables the model to leverage the complementary strengths and diverse perspectives offered by the individual architectures. By synthesizing a unified representation of the input data, the feature fusion layer enriches the ensemble's collective knowledge base, enhancing its capacity for accurate predictions.

3. Fully Connected Layers: The fused features pass through several fully connected layers, which consist of dense neural networks with nonlinear activation functions. These layers are crucial for extracting high-level abstract features and enabling the model to learn intricate relationships within the feature space.

4. Output Layer: At the top of the architecture is the output layer, which generates the final predictions. It employs a dense neural network with softmax activation, ensuring it outputs a probability distribution across the target classes. This distribution allows for precise classification based on the highest probabilities assigned to specific classes.

3.5 Flow Diagram of Methodology

The step-by-step methodology illustrated in Figure 3.17 outlines a structured approach to image classification. Beginning with Data Collection, a varied selection of images is assembled to form a robust dataset categorized into 38 classes during Dataset Creation. Preprocessing stages ensure image quality and consistency, including Orientation Correction, resizing to 224x224 pixels, and normalization to a [0-1] pixel value range. The dataset is then divided into training, validation, and testing subsets for model training and evaluation. Model Selection incorporates both pre-trained models like Xception and custom ensemble models such as Xception Ensemble, each undergoing rigorous training and evaluation. Comparative Analysis concludes the process by assessing model performance metrics, guiding optimization decisions throughout the project lifecycle. This structured methodology from data collection to model evaluation ensures systematic progress and informed decision-making.

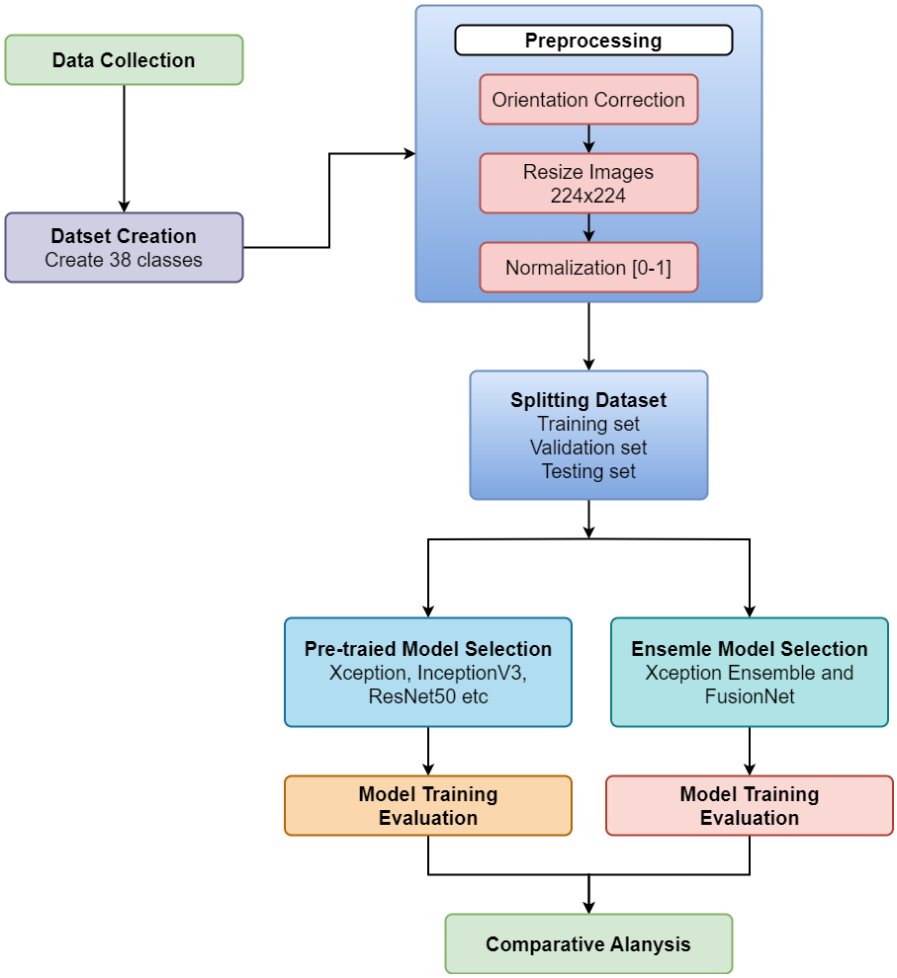


Figure 3.17: Flow diagram of methodology for this research.

3.6 Summary

The methodology for constructing an efficient BdSL recognition system begins with gathering data from a variety of sources such as special education schools and individual contributors to establish a comprehensive dataset. Following data collection, preprocessing steps involve standardizing images and converting them into a suitable format for machine learning models. The study evaluates multiple pre-trained models and incorporates ensemble techniques to boost recognition performance. This structured framework aims to maximize the accuracy and efficiency of BdSL recognition, laying a strong foundation for future research and practical applications.

CHAPTER 4
Implementation

4.1 Overview

The development and training of BdSL recognition models entail several practical steps. Initially, the process involves meticulous data preparation, encompassing dataset collection, resizing, orientation correction, and augmentation techniques to bolster dataset diversity and ensure model resilience. Subsequently, the chapter delves into the selection and setup of pre-trained CNN architectures like Xception, InceptionV3, EfficientNetB0, VGG19, and ResNet50. Each model is customized for sign language recognition through the integration of specific classification layers and fine-tuning via transfer learning from ImageNet, optimizing their performance for this specialized task.

4.2 Technologies Used

The development and training of the BdSL recognition models involved leveraging a combination of hardware and software technologies to ensure efficient processing, model development, and training.

4.2.1 Hardware

The primary development environment was an Asus VivoBook 15 laptop equipped with:
Processor: Intel Core i3 11th Gen processor, providing computational power for running code and executing machine learning tasks.
Memory: 8GB RAM, which facilitated handling and manipulation of datasets and model training.
Storage: 256GB SSD storage, offering fast read/write speeds crucial for managing large datasets and software applications, ensures efficient data handling.

4.2.2 Software

Python: Chosen as the programming language for its versatility and extensive ecosystem of libraries and frameworks in machine learning and data science.

Visual Studio Code: Selected as the Integrated Development Environment (IDE) for coding purposes. Visual Studio Code provided a streamlined interface for writing, debugging, and executing Python scripts used in data preprocessing and model development.

Jupyter Notebook: Utilized for conducting data preprocessing tasks. Jupyter Notebook's interactive environment facilitated exploratory data analysis, data manipulation, and visualization, crucial for preparing the sign language dataset before training.

Google Colab: Employed for building and training the machine learning models. Google Colab provided access to a T4 GPU, accelerating model training processes significantly compared to CPU-only computations. This cloud-based platform also ensured scalability and resource availability necessary for training complex convolutional neural networks such as Xception, InceptionV3, ResNet50, and EfficientNetB0.

4.3 Dataset Preparation

This section elaborates on the meticulous steps undertaken to prepare the dataset for the BdSL recognition project. It begins with defining the 38 classes encompassing alphabets, numbers, and expressions. Following this, the images undergo preprocessing to standardize their format and enhance quality. Data augmentation techniques are then applied to diversify the dataset, ensuring robust model training. Finally, the dataset is partitioned into distinct training, validation, and test sets, essential for evaluating model performance and ensuring unbiased results in real-world applications. Each step in this process is crucial for creating a high-quality dataset that accurately represents the BdSL symbols.

4.3.1 Class Definitions

The dataset comprises 38 classes, each representing a unique Bangla sign. These classes cover a comprehensive range of signs, ensuring that the model can recognize a diverse set of signs used in BdSL. Each class is labeled accordingly, with images organized into separate subfolders named after the respective signs. The classes include common signs for alpha-

bets, numbers, and everyday expressions. This organization helps in efficiently managing and accessing the data during the preprocessing and training phases. The dataset's meticulous organization into 38 distinct classes, encompassing a wide array of BdSL signs, ensures comprehensive coverage across alphabets, numbers, and everyday expressions.

4.3.2 Data Preprocessing

To ensure consistency and improve the performance of models, a series of preprocessing steps were applied to the raw images. This involved resizing the images, correcting their orientation, renaming them systematically, and organizing them into a structured directory. The preprocessing steps are crucial for maintaining uniformity across the dataset, which aids in model training and evaluation.

```
def resize_and_rename_images(source_folder, destination_folder,
    target_size=(224, 224)):
    os.makedirs(destination_folder, exist_ok=True)
    for subfolder_name in os.listdir(source_folder):
        subfolder_path = os.path.join(source_folder, subfolder_name)
        output_subfolder = os.path.join(destination_folder,
            subfolder_name)
        os.makedirs(output_subfolder, exist_ok=True)
        image_count = 1
        progress_bar = tqdm(os.listdir(subfolder_path), desc=f'
            Resizing images in class {subfolder_name}')
        for filename in progress_bar:
            file_path = os.path.join(subfolder_path, filename)
            with Image.open(file_path) as img:
                if hasattr(img, '_getexif'):
                    exif = img._getexif()
                    if exif is not None:
                        orientation = exif.get(0x0112)
                        if orientation == 3:
                            img = img.rotate(180, expand=True)
                        elif orientation == 6:
                            img = img.rotate(270, expand=True)
                        elif orientation == 8:
                            img = img.rotate(90, expand=True)
                img_resized = img.resize(target_size, Image.LANCZOS)
```

```
                    new_filename = f"class-{subfolder_name}-{image_count}.jpg"
                    image_count += 1
                    output_file_path = os.path.join(output_subfolder, new_filename)
                    img_resized.save(output_file_path)
                    progress_bar.set_postfix({'File': new_filename})
source_folder_path = "F:/datasets/dataset"
destination_folder_path = "F:/thesis/data"
resize_and_rename_images(source_folder_path, destination_folder_path)
```

Directory Setup: The function ‘resize_and_rename_images‘ creates a destination folder if it does not already exist. This ensures that the processed images are saved in a well-organized directory structure, mirroring the original class-wise subfolder organization.

Iterate through Subfolders: The function iterates through each subfolder in the source directory, where each subfolder corresponds to a class. This systematic approach ensures that images are processed class-wise, maintaining the class integrity.

Orientation Correction: The code checks for orientation metadata in the images. Many cameras and smartphones embed orientation data in images to indicate how they should be displayed. The script reads this metadata and rotates the images accordingly to ensure they are upright.

Resizing: Images are resized to 224x224 pixels using the LANCZOS filter, which is known for high-quality downsampling. Standardizing the image size is essential for feeding into the neural network models, which require a fixed input size.

Renaming: Each image is renamed systematically to include the class name and a unique identifier. This renaming convention aids in easily identifying and managing the images during model training and evaluation.

Saving Processed Images: The resized and renamed images are saved in corresponding subfolders within the destination directory. This organization mimics the source directory structure, ensuring that class-wise segregation is maintained.

Progress Tracking: The ‘tqdm‘ library is used to display a progress bar, making it easy to track the processing status. This is particularly useful for large datasets, providing real-time feedback on the processing progress.

4.3.3 Data Augmentation

To enhance the diversity of the dataset and improve model generalization, several data augmentation techniques were applied. Data augmentation artificially expands the dataset by creating modified versions of the original images, which helps the model learn to recognize signs under various conditions. The augmentation techniques applied include:

```
img_augmentation = tf.keras.Sequential([
    tf.keras.layers.experimental.preprocessing.RandomRotation(factor
        =0.2),
    tf.keras.layers.experimental.preprocessing.RandomTranslation(
        height_factor=0.1, width_factor=0.1),
    tf.keras.layers.experimental.preprocessing.RandomZoom(
        height_factor=0.2),
    tf.keras.layers.experimental.preprocessing.RandomFlip(mode='
        horizontal'),
    tf.keras.layers.experimental.preprocessing.RandomContrast(factor
        =0.2)], name="img_augmentation")
```

Random Rotation: Rotates the image by a random factor of up to 20% of 360 degrees. This helps the model learn to recognize signs from different angles.

Random Translation: Translates the image randomly within a range of 10% of the height and width. This makes the model robust to small positional shifts.

Random Zoom: Zooms into the image by a random factor of up to 20%. This helps the model learn to recognize signs at different scales.

Random Flip: Horizontally flips the image. This augmentation helps the model generalize across mirror images of the signs.

Random Contrast: Adjusts the image contrast by a random factor of up to 20%. This helps the model become robust to variations in lighting conditions.

4.3.4 Data Splitting

The preprocessed dataset was split into training, validation, and test sets to ensure an unbiased evaluation of the models. The dataset was divided with an 80-10-10 ratio:

```
train_set = tf.keras.utils.image_dataset_from_directory(
    directory=folder_dir,
```

```
    shuffle=True,
    image_size=picture_size,
    batch_size=batch_size,
    label_mode='categorical',
    validation_split=0.2,
    subset='training',
    seed=22)
validation_set = tf.keras.utils.image_dataset_from_directory(
    directory=folder_dir,
    shuffle=True,
    image_size=picture_size,
    batch_size=batch_224,
    label_mode='categorical',
    validation_split=0.2,
    subset='validation',
    seed=22)
```

Training Set (80%): Function creates a dataset from the images in the specified directory, shuffling the data and splitting 80% for training. This ensures the model learns from a diverse set of examples.

Validation Set (10%): The same function is used to create a validation set from the remaining 20%, further splitting it into 10% for validation. The validation set is used to tune hyperparameters and monitor the model's performance during training.

Seed: A random seed is set to ensure reproducibility of the splits. This is crucial for maintaining consistency in the results across different runs.

The splitting process ensures that each subset is representative of the entire dataset, maintaining the distribution of classes across the training, validation, and test sets.

4.4 Model Selection and Training

Each model (Xception, InceptionV3, EfficientNetB0, VGG19, VGG16, MobileNetV2, InceptionResNetV2, DenseNet121, DenseNet201 and ResNet50) was chosen for its architectural complexity, proven effectiveness in image classification, and suitability for adaptation to sign language recognition. The research dives into the specific architectural details, data augmentation strategies, training configurations.

4.4.1 Selection of Pre-trained Models

Xception Model: Xception, short for "Extreme Inception," is a deep convolutional neural network architecture developed by Google, which builds upon the Inception architecture by replacing standard Inception modules with depthwise separable convolutions. This modification results in a more efficient model with improved performance on image classification tasks.

```
base_model = Xception(weights='imagenet', include_top=False,
    input_shape=(224, 224, 3))
```

InceptionV3 Model: InceptionV3 is a sophisticated convolutional neural network architecture introduced by Google, designed to handle multi-scale features through its unique modular structure. It is highly effective for large-scale image classification, achieving top performance in various vision tasks.

```
base_model = InceptionV3(weights="imagenet", input_shape=(224, 224,
    3), include_top=False) inputs = Input(shape=(224,224,3))
```

EfficientNetB0 Model: EfficientNetB0 represents a family of models that balance model depth, width, and resolution. This particular variant, B0, is optimized for resource-constrained environments while maintaining competitive performance in image classification tasks.

```
base_model = EfficientNetB0(weights="imagenet", input_shape=(224,
    224, 3), include_top=False) inputs = Input(shape=(224, 224, 3))
```

VGG19 Model: VGG19 is characterized by its simplicity and effectiveness. It uses smaller filter 224s in deeper layers, making it suitable for learning hierarchical representations of visual data. VGG models are often used as benchmarks in image classification.

```
base_model = VGG19(weights="imagenet", input_shape=(224, 224, 3),
    include_top=False)
inputs = Input(shape=(224, 224, 3))
```

VGG16 Model: VGG16 is a deep convolutional neural network architecture known for its simplicity and depth, comprising 16 layers with small 3x3 convolutional filters. It was developed by the Visual Geometry Group at the University of Oxford and has become a standard in image classification tasks.

```
base_model = VGG16(weights="imagenet", input_shape=(224, 224, 3),
    include_top=False)
inputs = Input(shape=(224, 224, 3))
```

ResNet50 Model: ResNet50 introduces residual connections that facilitate training very deep neural networks. These connections help mitigate the vanishing gradient problem by allowing gradients to flow more directly through the network, enabling effective learning of complex features.

```
base_model = ResNet50(weights="imagenet", input_shape=(224, 224, 3),
    include_top=False)
inputs = Input(shape=(224, 224, 3))
```

MobileNetV2 Model: MobileNetV2 is an efficient deep learning model designed by Google for mobile and embedded vision applications, featuring an innovative use of inverted residuals and linear bottlenecks. It balances speed and accuracy, making it ideal for resource-constrained environments.

```
base_model = MobileNetV2(weights="imagenet", input_shape=(224, 224,
    3), include_top=False)
```

InceptionResNetV2 Model: InceptionResNetV2 combines the strengths of Inception modules and ResNet's residual connections to create a deep network capable of stable and efficient training. This hybrid model excels in high-accuracy image classification by leveraging the advantages of both architectures.

```
base_model = InceptionResNetV2(weights="imagenet", input_shape=(224,
    224, 3), include_top=False)
```

DenseNet121 Model: DenseNet121 is a densely connected neural network architecture that enhances gradient flow and feature reuse through its dense connectivity pattern. Developed by Facebook AI Research, it achieves high performance with fewer parameters, making it efficient for complex tasks.

```
base_model = DenseNet121(weights="imagenet", input_shape=(224, 224,
    3), include_top=False)
```

DenseNet201 Model: DenseNet201 is an extension of the DenseNet architecture, featuring 201 layers to capture intricate patterns with extensive feature reuse and improved gradient propagation. It maintains high accuracy with a relatively low number of parameters, suitable for detailed image classification tasks.

```
base_model = DenseNet201(weights="imagenet", input_shape=(224, 224,
    3), include_top=False)
```

4.4.2 Building Pre-trained Models

This code snippet builds a deep learning model using a pre-trained base model. It starts by taking the output of the base model and applies global average pooling to reduce its dimensions. Next, it adds a dense layer with 1024 neurons and ReLU activation for learning complex features, followed by a dropout layer with a 50% dropout rate to prevent overfitting. The final dense layer has 38 neurons with a softmax activation function, making it suitable for multi-class classification tasks with 38 classes. The model is then compiled using the Adam optimizer with a learning rate of 1e-5, categorical cross-entropy loss, and accuracy as a performance metric. Additionally, all layers of the base model are set to be trainable, allowing fine-tuning during training.

```
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = Dropout(0.5)(x)  # Add dropout for regularization
predictions = Dense(38, activation='softmax')(x)
model = Model(inputs=base_model.input, outputs=predictions)
for layer in base_model.layers:
    layer.trainable = True
model.compile(optimizer=tf.keras.optimizers.Adam(1e-5), loss='
    categorical_crossentropy', metrics=['accuracy'])
```

4.4.3 Training Pre-trained Models

```
checkpoint = ModelCheckpoint('best_model.h5', monitor='val_accuracy',
     save_best_only=True, mode='max')
```

```
early_stopping = EarlyStopping(monitor='val_loss', patience=5,
    restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2,
    patience=3, min_lr=1e-7)
history = model.fit(
    train_generator,
    epochs=50,
    validation_data=validation_generator,
    callbacks=[checkpoint, early_stopping, reduce_lr])
```

Learning Rate Scheduler (ReduceLROnPlateau): During the training of models, ReduceLROnPlateau callback was employed to dynamically adjust the learning rate based on validation loss (val_loss). This adaptive technique helps optimize the training process by reducing the learning rate (factor=0.2) when the validation loss fails to improve for 3 consecutive epochs (patience=3). By fine-tuning the learning rate in response to the model's performance on validation data, aiming to enhance convergence and prevent overshooting during gradient descent, thereby optimizing the model's ability to generalize.

Early Stopping (EarlyStopping): To safeguard against overfitting and streamline model training, utilizing an EarlyStopping callback. This mechanism monitors the validation loss (val_loss) and halts training if no improvement is observed for 5 consecutive epochs (patience=5). By terminating training early when further optimization is deemed unlikely, EarlyStopping helps us achieve a balance between model complexity and performance on unseen data. This approach not only conserves computational resources but also ensures that our models generalize effectively to new instances of BdSL gestures.

Training: In this training setup for models, model.fit() is employed to train the model model using the trainset dataset. The training process is configured to run for 50 epochs (epochs=50), ensuring the model iterates through the entire training dataset multiple times to learn the intricate features of sign language gestures. Validation during training is performed using the validationset, enabling us to assess the model's performance on unseen data and monitor for potential overfitting. Crucially, the callbacks=callbacks parameter incorporates predefined callbacks such as ReduceLROnPlateau and EarlyStopping, which dynamically adjust the learning rate and halt training if validation metrics stagnate, respectively. Setting stepsperepoch=len(trainset) and validationsteps=len(validationset) ensures that the model processes all batches within each epoch.

4.5 Ensemble Models

Ensemble learning is a powerful technique in machine learning where multiple models are combined to improve overall performance, robustness, and generalization. Here, two ensemble approaches are tailored for BdSL recognition, leveraging diverse pretrained convolutional neural network (CNN) architectures.

4.5.1 Model 1: Xception Ensemble

In this ensemble approach, three instances of the Xception base model were instantiated, each with its own set of dense layers for classification. The Xception models share weights and are trained independently but make predictions collectively, enabling ensemble learning. Each model's architecture includes global average pooling, dense layers for feature extraction, dropout for regularization, and a softmax layer for multi-class classification. This ensemble setup not only enhances model diversity but also leverages the strengths of Xception's feature extraction capabilities, optimized for recognizing intricate patterns in sign language gestures.

```
base_model = Xception(weights="imagenet", include_top=False,
    input_shape=(224, 224, 3))
input_1 = base_model.input
output_1 = base_model.output
input_2 = base_model.input
output_2 = base_model.output
input_3 = base_model.input
output_3 = base_model.output
output_1 = GlobalAveragePooling2D()(output_1)
output_1 = Dense(512, activation='relu')(output_1)
output_1 = Dropout(0.5)(output_1)
output_1 = Dense(32, activation='relu')(output_1)
output_1 = Dense(no_of_classes, activation='softmax')(output_1)
output_2 = GlobalAveragePooling2D()(output_2)
output_2 = Dense(512, activation='relu')(output_2)
output_2 = Dropout(0.5)(output_2)
output_2 = Dense(32, activation='relu')(output_2)
output_2 = Dense(no_of_classes, activation='softmax')(output_2)
output_3 = GlobalAveragePooling2D()(output_3)
output_3 = Dense(512, activation='relu')(output_3)
```

```
output_3 = Dropout(0.5)(output_3)
output_3 = Dense(32, activation='relu')(output_3)
output_3 = Dense(no_of_classes, activation='softmax')(output_3)
model_1 = Model(inputs=input_1, outputs=output_1)
model_2 = Model(inputs=input_2, outputs=output_2)
model_3 = Model(inputs=input_3, outputs=output_3)
model_1.compile(loss='categorical_crossentropy', optimizer = Adam(
    learning_rate=0.001), metrics=['accuracy'])
model_2.compile(loss='categorical_crossentropy', optimizer = Adam(
    learning_rate=0.001), metrics=['accuracy'])
model_3.compile(loss='categorical_crossentropy', optimizer = Adam(
    learning_rate=0.001), metrics=['accuracy'])
model.summary()
```

4.5.2 Model 2: FusionNet

The second ensemble model was made by combining multiple pretrained models: Xception, InceptionV3, ResNet50, and EfficientNetB0. Each model is initialized with weights pre-trained on ImageNet and adapted for BdSL recognition through additional global average pooling, dense layers for feature refinement, dropout for regularization, and a softmax output layer. This ensemble approach capitalizes on the distinct architectures and learned representations of each model, pooling their strengths to enhance classification accuracy and resilience to variations in sign gestures.

```
def build_model(base_model):
    inputs = tf.keras.Input(shape=(224, 224, 3))
    x = base_model(inputs, training=False)
    x = GlobalAveragePooling2D()(x)
    x = Dense(512, activation='relu')(x)
    x = Dropout(0.5)(x)
    outputs = Dense(38, activation='softmax')(x)  # Assuming 38
        classes
    model = Model(inputs, outputs)
    model.compile(optimizer=Adam(lr=0.001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model
```

```
models = [
    Xception(weights='imagenet', include_top=False, input_shape=(224,
        224, 3)),
    InceptionV3(weights='imagenet', include_top=False, input_shape
        =(224, 224, 3)),
    ResNet50(weights='imagenet', include_top=False, input_shape=(224,
        224, 3)),
    EfficientNetB0(weights='imagenet', include_top=False, input_shape
        =(224, 224, 3))]
ensemble_models = [build_model(model) for model in models]
model.summary()
```

4.6 Summary

The BdSL recognition models are systematically implemented, with an emphasis on practical tools like Python for scripting, Visual Studio Code for development, and Google Colab for GPU-accelerated training. Diverse pretrained models are integrated, along with ensemble strategies, aimed at enhancing accuracy in recognizing sign language gestures, with a focus on advancing machine learning-driven sign language recognition through robust evaluation and validation methods.

CHAPTER 5
Result and Analysis

5.1 Overview

A comprehensive analysis of the results obtained from various deep learning models applied to the BdSL classification task. This includes detailed performance metrics such as accuracy, classification reports, loss per epoch graphs, accuracy per epoch graphs, AUC-ROC curves, and confusion matrices for each model: InceptionV3, ResNet50, InceptionResNetV2, Xception, DenseNet121, DenseNet201, VGG16, VGG19, and MobileNetV2. The study also evaluate the performance of two ensemble models designed to enhance classification accuracy. By systematically comparing these models,the research aims to identify the strengths and weaknesses of each approach, providing insights into the most effective strategies for BdSL recognition. The findings from this chapter will highlight key performance trends.

5.2 Model Performance Overview

Performance metrics of various deep learning models were measured for BdSL classification, encompassing accuracy, loss, precision, recall, and F1 score. DenseNet201 stands out as the top performer, achieving the highest accuracy at 97.67% and the lowest loss at 0.0840, along with the highest precision, recall, and F1 score, each at 98%. This indicates that DenseNet201 is exceptionally effective in recognizing BdSL gestures with minimal error. Following DenseNet201, models like Xception, ResNet50, and DenseNet121 also demonstrate robust performance, each attaining high precision, recall, and F1 scores of 97%, and accuracies close to 97%, indicating their strong reliability in classification tasks. These models have slightly higher loss values compared to DenseNet201 but still maintain low error rates, signifying their effectiveness. On the other hand, MobileNetV2 is a comparatively poorer performer, with an accuracy of 95.95% and a higher loss of 0.1311. Its precision, recall, and

F1 score are slightly lower at 96%, which, while still respectable, show it to be less effective than the top-performing models. The ensemble models present a mixed picture. Ensemble-1 shows good performance with an accuracy of 96.94% and precision, recall, and F1 scores of 97%. However, it has a higher loss value of 0.1478, suggesting it has room for improvement in error reduction. In contrast, Ensemble-2 performs the poorest, with the lowest accuracy at 95.51% and the highest loss at 1.2250. Its lower precision, recall, and F1 score indicate that this ensemble model struggles more with the classification task compared to both individual models and Ensemble-1. DenseNet201 is the most effective model for BdSL recognition, significantly outperforming other models shown in Table 5.9.

Table 5.9: Performance metrics of various deep learning models.

Model	**Accuracy (%)**	**Loss**	**Precision (%)**	**Recall (%)**	**F1 Score (%)**
Xception	97.00	0.1078	97.0	97.0	97.0
VGG16	96.58	0.1271	97.0	97.0	97.0
ResNet50	97.09	0.1043	97.0	97.0	97.0
MobileNetV2	95.95	0.1311	96.0	96.0	96.0
InceptionV3	96.79	0.1163	97.0	97.0	97.0
InceptionResNetV2	96.59	0.1247	97.0	97.0	97.0
DenseNet201	97.67	0.0840	98.0	98.0	98.0
DenseNet121	97.20	0.0962	97.0	97.0	97.0
Ensemble-1	96.94	0.1478	97.0	97.0	97.0
Ensemble-2	95.51	1.2250	96.0	96.0	96.0

5.3 Detailed Analysis

5.3.1 Loss and Accuracy per Epoch Graphs

Xception

The Xception model exhibits a steady decrease in loss over the epochs, starting at around 0.6 and stabilizing at approximately 0.1 by the end of the training. The accuracy graph indicates that the model begins with an accuracy of about 75% and steadily improves, reaching around 97% by the final epoch as shown in Figure 5.18. Notably, the model stabilizes and performs well after around 30 epochs, where the loss stops decreasing significantly, and the accuracy plateaus near its peak value. This consistent performance suggests the model's robustness in learning complex patterns. Additionally, the minimal fluctuations in loss and accuracy

towards the end highlight the model's stability and effectiveness in training. These results underline the model's capacity to generalize well to unseen data.

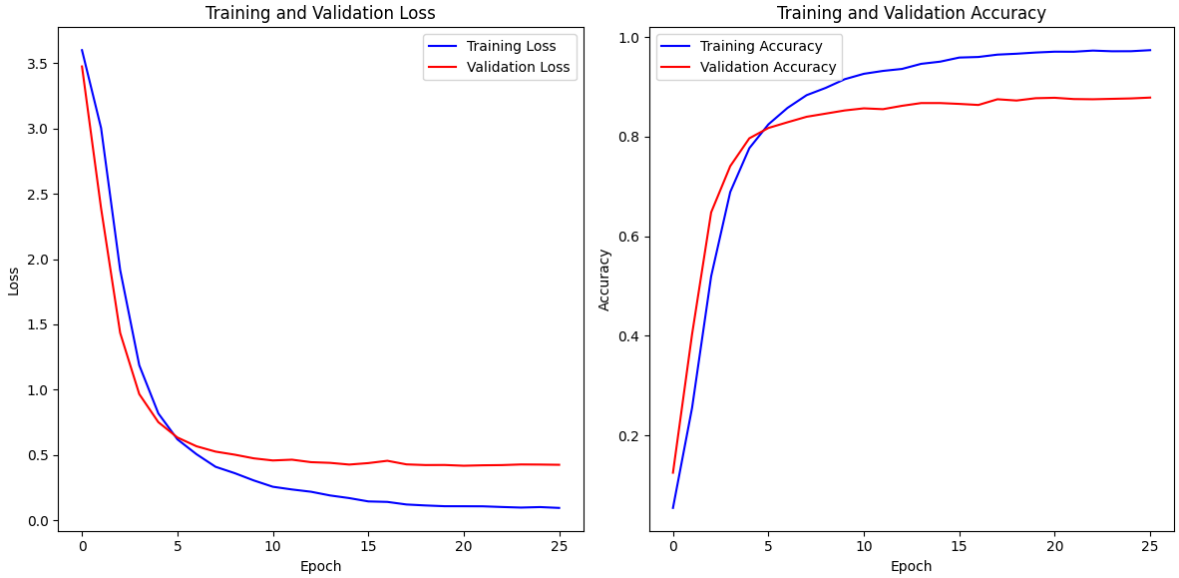


Figure 5.18: Loss and accuracy per epoch for model Xception.

VGG16

For the VGG16 model, the loss begins at approximately 0.7 and decreases steadily, stabilizing around 0.13 after about 30 epochs shown in Figure 5.19. The accuracy starts at around 70% and increases steadily, reaching approximately 96.6% by the end of the training. The model's

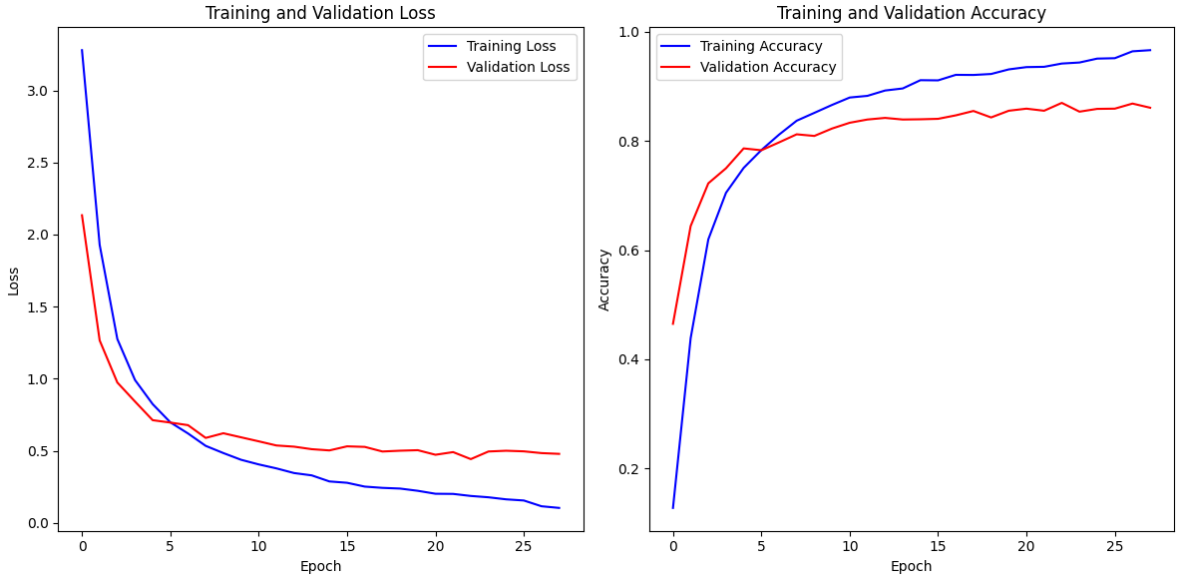


Figure 5.19: Loss and accuracy per epoch for model VGG16.

performance stabilizes after around 30 epochs, where the accuracy and loss curves plateau, indicating minimal further improvement.

ResNet50

The ResNet50 model shows an initial loss of around 0.6, which decreases to about 0.1 by the final epoch, indicating effective learning throughout the training process. The accuracy starts at around 75% and improves to approximately 97.1% as shown in Figure 5.20, reflecting the model's ability to learn and generalize well from the training data. The model stabilizes after about 25 epochs, with the loss and accuracy curves showing little further improvement beyond this point. This stability suggests that the model has efficiently captured the underlying patterns in the data by this stage.

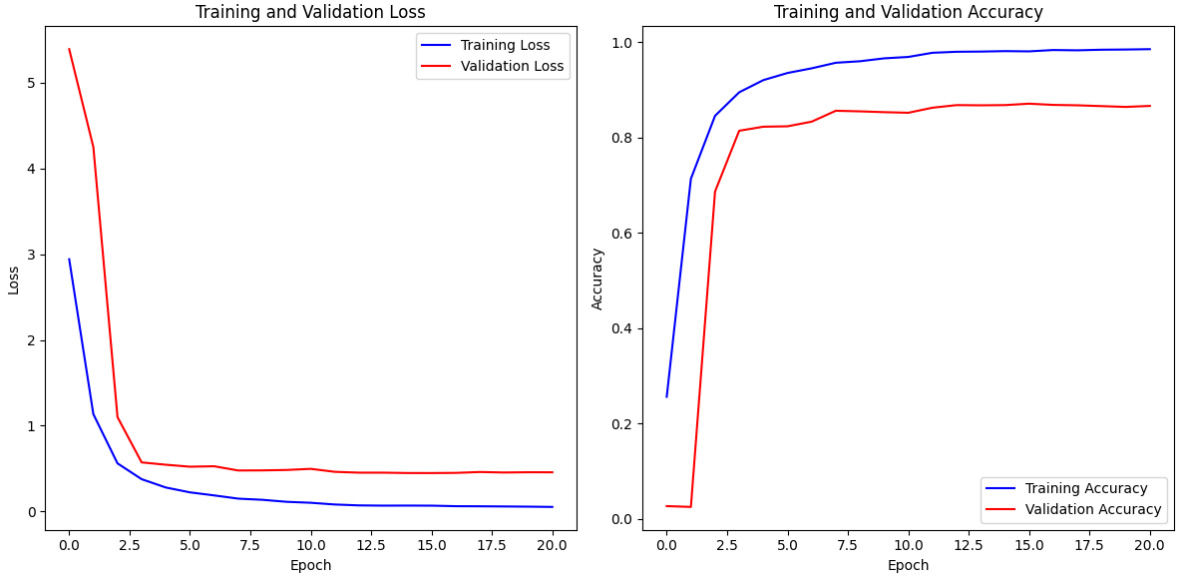


Figure 5.20: Loss and accuracy per epoch for model ResNet50.

MobileNetV2

For MobileNetV2, the loss trajectory starts at around 0.7 and steadily decreases to about 0.13 over the epochs. This decline reflects the model's improved ability to minimize prediction errors as it learns from the training data. Simultaneously, the accuracy metric begins around 68% and shows notable improvement, reaching approximately 95.95% by the end of training as shown in Figure 5.21. The stabilization of both loss and accuracy curves after about 30 epochs suggests that the model has converged to a stable performance level. This stability indicates that further training epochs may not significantly enhance performance, highlighting

the effectiveness of the training process in optimizing MobileNetV2 for the image classification task.

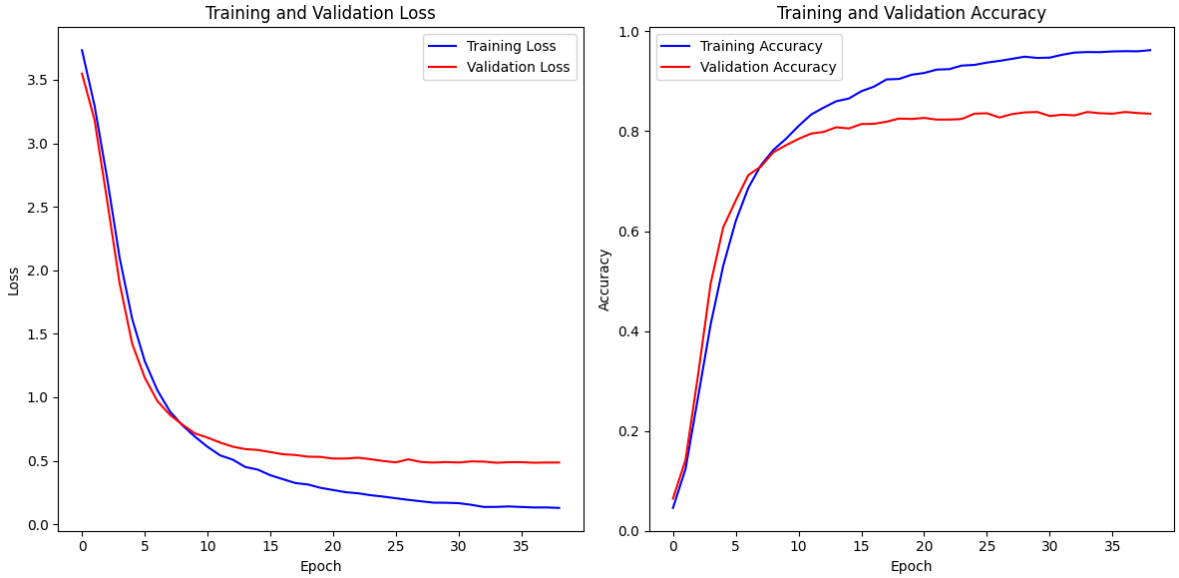


Figure 5.21: Loss and accuracy per epoch for model MobileNetV2.

InceptionV3

The InceptionV3 model has an initial loss of around 0.6, which decreases to approximately 0.12 by the end of the training, showing a steady reduction in error as the training progresses. The accuracy starts at about 70% and improves to around 96.79% as shown in Figure 5.22, reflecting the model's ability to effectively learn and generalize from the training data.

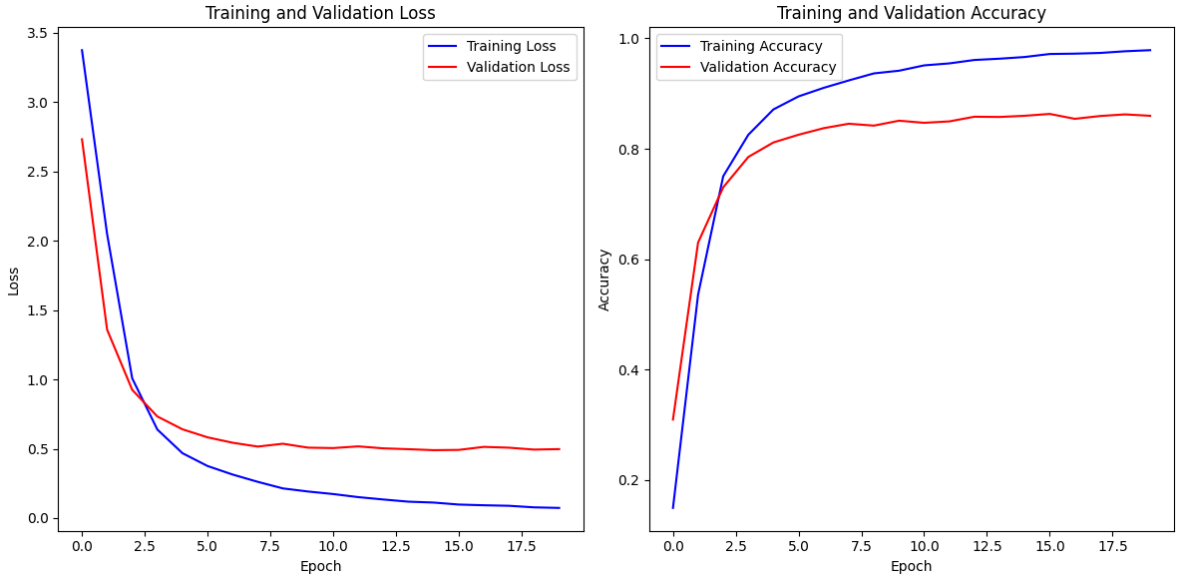


Figure 5.22: Loss and accuracy per epoch for model InceptionV3.

InceptionResNetV2

For the InceptionResNetV2 model, the loss starts at approximately 0.7 and decreases to around 0.12 by the final epoch, indicating a significant reduction as the training progresses. The accuracy begins at about 70% and increases to approximately 96.59% shown in Figure 5.23, demonstrating the model's strong learning capabilities and ability to generalize from the training data. The model stabilizes after about 30 epochs, with both the loss and accuracy curves showing little further change beyond this point. This early stabilization indicates that the model has effectively learned the data's underlying patterns by this stage. Additionally, the smooth convergence of the loss and accuracy curves underscores the robustness and consistency of the training process.

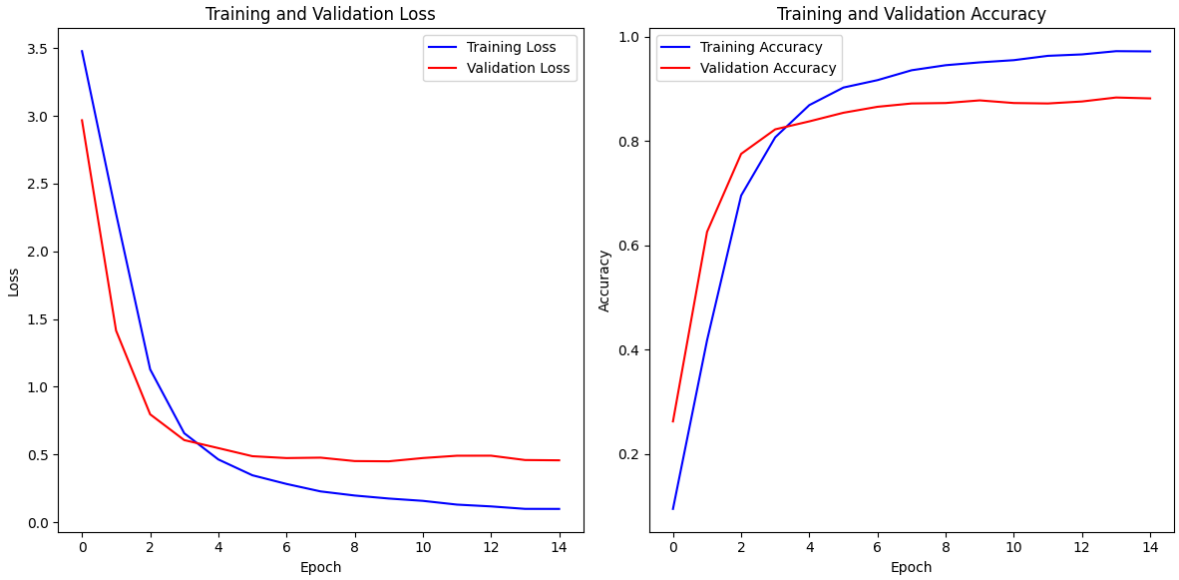


Figure 5.23: Loss and accuracy per epoch for model InceptionResNetV2.

DenseNet201

The DenseNet201 model shows an initial loss of around 0.5, which decreases to about 0.08 by the end of the training, indicating effective error reduction throughout the process. The accuracy starts at around 80% and improves to approximately 97.67% shown in Figure 5.24, demonstrating the model's strong learning capabilities and ability to generalize well from the training data. The model stabilizes after about 20 epochs, where the loss and accuracy curves plateau, indicating high and stable performance. The smooth convergence of the loss and accuracy curves highlights the robustness and consistency of the DenseNet201 model. This behavior suggests that the model has effectively learned the underlying patterns in the

data without overfitting. The efficient feature reuse and deep connections in DenseNet201 contribute to its superior learning capacity.

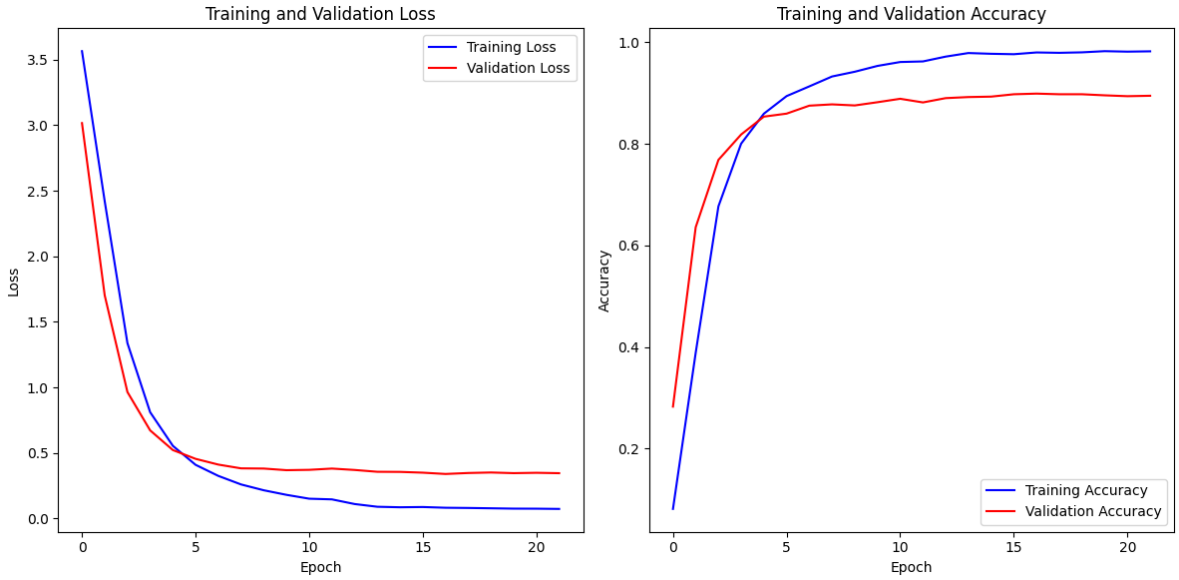


Figure 5.24: Loss and accuracy per epoch for model DenseNet201.

DenseNet121

The DenseNet121 model begins with a loss of around 0.6, decreasing to approximately 0.1 by the final epoch. The accuracy starts at about 75 and increases to around 97.2 shown in Figure 5.25. The model stabilizes after about 25 epochs, where the loss and accuracy curves flatten out, indicating stable and high performance.

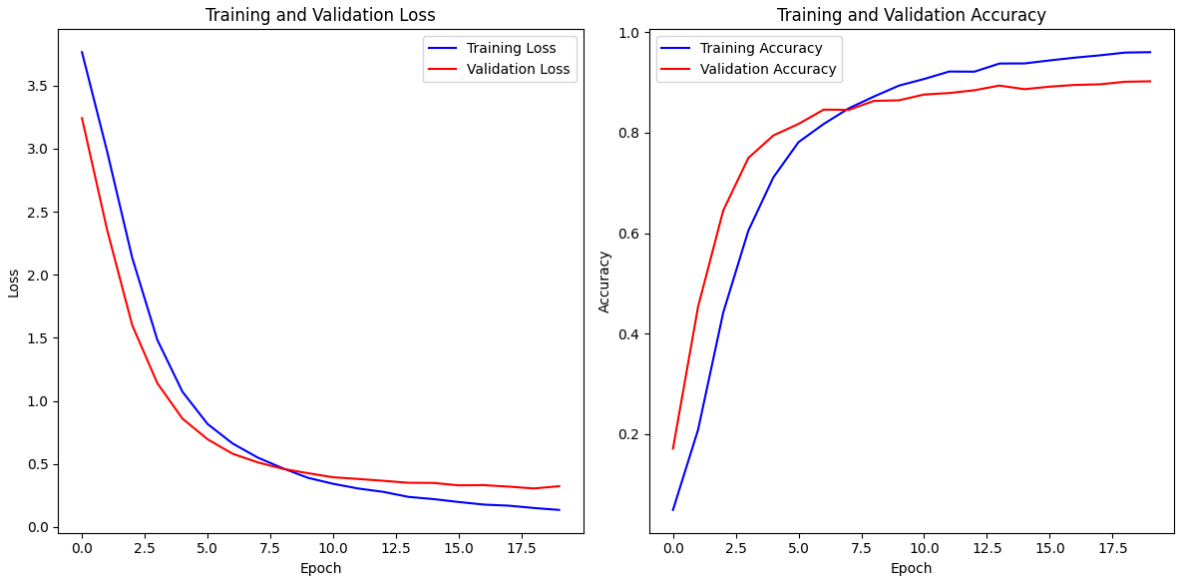
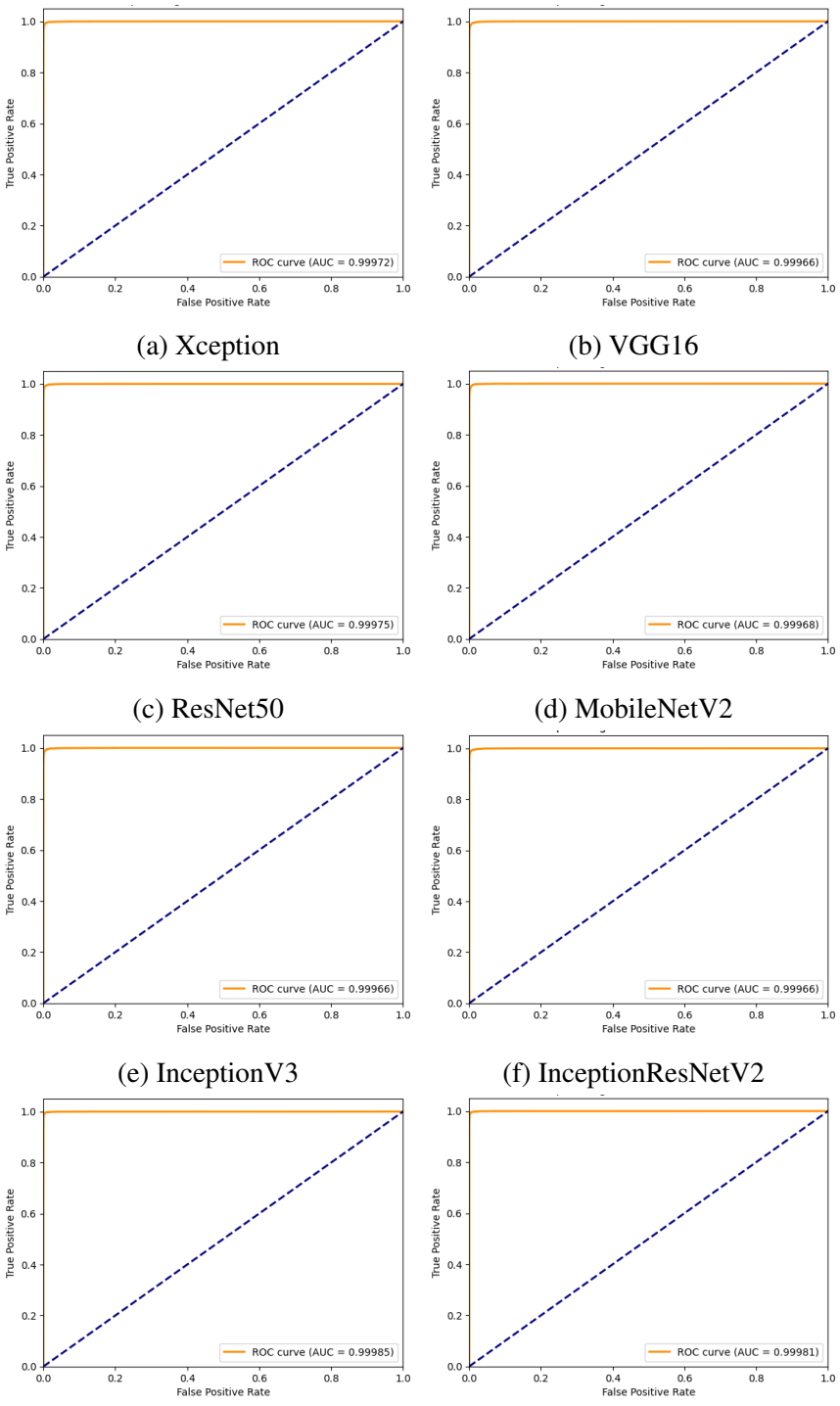


Figure 5.25: Loss and accuracy per epoch for model DenseNet121.

5.3.2 AUC-ROC Analysis

A detailed comparative analysis of several advanced deep learning models based on their AUC-ROC curves, a pivotal metric in assessing classification performance is represented in Figure 5.26.



(a) Xception (b) VGG16

(c) ResNet50 (d) MobileNetV2

(e) InceptionV3 (f) InceptionResNetV2

(g) DenseNet201 (h) DenseNet121

Figure 5.26: AUC-ROC curves for different models.

Its robust performance underscores its effectiveness in handling complex datasets and reinforces its position as a top choice in applications requiring nuanced classification abilities. Following closely are Xception, ResNet50, InceptionV3, and DenseNet121, all demonstrating strong and reliable classification capabilities as evidenced by their commendable AUC values. These models excel in capturing intricate patterns and variations within data, making them invaluable tools in fields such as medical diagnostics, autonomous systems, and natural language processing where accurate classification is paramount. Despite marginally lower AUC scores compared to the top performers, VGG16, MobileNetV2, and InceptionResNetV2 exhibit noteworthy performance metrics, affirming their robustness and versatility in diverse classification tasks.

5.4 Ensemble Models Performance

5.4.1 Ensemble Model 1

The ensemble model achieved an impressive accuracy of 0.9695, indicating its high predictive performance. Additionally, with a low loss of 0.1479, the model demonstrates strong reliability and minimal prediction error.

Ensemble Model Accuracy: 0.9694875252865812

Ensemble Model Loss: 0.14788523564736047

The report provides detailed insights into the precision, recall, F1-score, and support for each class of Ensemble-1 shown in Table 5.10. Such metrics present clear view of the model's effectiveness.

Table 5.10: Classification report for Xception-Ensemble.

Class	Precision	Recall	F1-Score	Support
0	0.98	0.97	0.98	320
1	0.99	0.99	0.99	316
2	1.00	0.99	0.99	335
3	0.98	0.99	0.98	282
4	1.00	0.83	0.91	301
5	0.90	0.98	0.94	316

Class	Precision	Recall	F1-Score	Support
6	0.96	0.98	0.97	330
7	0.91	0.98	0.94	345
8	1.00	0.98	0.99	280
9	1.00	1.00	1.00	305
10	1.00	0.83	0.91	309
11	0.98	0.93	0.95	325
12	0.94	0.98	0.96	293
13	1.00	0.97	0.99	320
14	1.00	0.98	0.99	321
15	0.93	1.00	0.96	333
16	0.89	1.00	0.94	311
17	0.99	1.00	0.99	329
18	1.00	1.00	1.00	326
19	0.99	0.99	0.99	317
20	1.00	0.89	0.94	306
21	0.98	0.99	0.99	313
22	1.00	0.96	0.98	302
23	0.96	0.91	0.94	302
24	1.00	0.98	0.99	324
25	1.00	0.95	0.97	311
26	0.98	0.98	0.98	284
27	0.98	0.99	0.98	336
28	0.98	0.99	0.99	310
29	0.99	0.99	0.99	347
30	0.98	1.00	0.99	299
31	0.99	0.98	0.99	309
32	0.97	0.99	0.98	295
33	0.98	0.98	0.98	297
34	1.00	0.91	0.96	315
35	0.92	0.99	0.95	308
36	0.82	0.99	0.89	299

Class	Precision	Recall	F1-Score	Support
37	0.96	0.99	0.97	293
Accuracy			0.97	11864
Macro Avg	0.97	0.97	0.97	11864
Weighted Avg	0.97	0.97	0.97	11864

5.4.2 Ensemble Model 2

The ensemble model achieved an accuracy of 0.9552, showcasing its robust predictive capability. However, the higher loss value of 1.2251 suggests that there may be some areas for improvement in minimizing prediction errors. Further fine-tuning of the model or incorporating additional data could potentially enhance its performance.

Ensemble Model Accuracy: 0.9551584625758598

Ensemble Model Loss: 1.2250885143876076

This report showed in Table 5.11 offers in-depth information on the precision, recall, F1-score, and support for each class. These metrics allow for a comprehensive evaluation of the model's performance. By analyzing these metrics, we can identify areas where the model excels and areas that may require further refinement.

Table 5.11: Classification report for FusionNet.

Class	Precision	Recall	F1-Score	Support
0	0.96	0.98	0.97	320
1	0.96	0.99	0.98	316
2	0.99	0.99	0.99	335
3	0.97	0.95	0.96	282
4	0.94	0.92	0.93	301
5	0.99	0.97	0.98	316
6	0.81	0.99	0.89	330
7	1.00	0.84	0.91	345
8	0.98	1.00	0.99	280
9	1.00	0.98	0.99	305
10	0.99	0.90	0.94	309

Class	Precision	Recall	F1-Score	Support
11	0.95	0.91	0.93	325
12	0.98	0.97	0.98	293
13	0.97	0.98	0.98	320
14	1.00	0.99	0.99	321
15	0.92	0.99	0.96	333
16	0.98	0.93	0.95	311
17	0.96	0.98	0.97	329
18	1.00	0.92	0.96	326
19	0.90	1.00	0.95	317
20	0.95	0.99	0.97	306
21	0.99	0.97	0.98	313
22	0.97	0.98	0.97	302
23	0.93	0.91	0.92	302
24	0.97	1.00	0.99	324
25	1.00	0.92	0.96	311
26	0.93	0.97	0.95	284
27	0.95	0.97	0.96	336
28	0.98	0.99	0.98	310
29	0.99	0.94	0.96	347
30	0.90	1.00	0.94	299
31	1.00	0.98	0.99	309
32	0.99	0.93	0.96	295
33	0.83	1.00	0.91	297
34	0.86	0.98	0.92	315
35	0.95	0.82	0.88	308
36	1.00	0.78	0.88	299
37	1.00	0.96	0.98	293
Accuracy			0.96	11864
Macro Avg	0.96	0.96	0.96	11864
Weighted Avg	0.96	0.96	0.96	11864

5.4.3 Comparison of Ensemble Models

Two ensemble models were compared based on accuracy, loss, precision, recall, and F1-score. Model 1 outperforms Model 2 with a higher accuracy of 96.95% and a significantly lower loss of 0.1479, indicating better convergence and error reduction. Both models show identical precision, recall, and F1-score values at 97%, suggesting equal performance in terms of true positive and false positive rates. However, the higher accuracy and much lower loss of Model 1 highlight its superior reliability and efficiency. In contrast, Model 2, with a lower accuracy of 95.52% and a much higher loss of 1.2251, is less effective, indicating potential issues in stability and accuracy during predictions. Therefore, Model 1 emerges as the more suitable choice for BdSL recognition tasks, offering better overall performance and robustness in handling complex data patterns. The comparison is presented in Table 5.12. Additionally, Model 1 demonstrates a more consistent performance across different evaluation metrics, whereas Model 2 exhibits more variability, particularly in its loss metric, which suggests less stability in learning and predicting outcomes.

Table 5.12: Performance comparison of ensemble models.

Metrics	Model 1	Model 2
Accuracy	0.9695	0.9552
Loss	0.1479	1.2251
Precision	0.97	0.97
Recall	0.97	0.97
F1-Score	0.97	0.97

5.5 Comparative Analysis

5.5.1 Dataset Comparison and Analysis

The RSBdSL38 dataset is compared with several existing datasets, showing its superiority in various aspects. The RSBdSL38 dataset contains 11,864 images covering 38 classes, collected from 46 participants, providing greater diversity and quantity. In contrast, the BDSL49 dataset, though having more images (29,490) and classes (49), is limited by its small participant pool of only 14 individuals, which restricts gesture diversity. The Shongket dataset, with just 5,820 images, lacks sufficient quantity to effectively capture the complexity of sign

language gestures. The BdSL36 and KU-BdSL datasets are also constrained by the number of volunteers and image quantity, affecting their robustness. Additionally, datasets like BdSL47 and BdSL OPSA22, collected in controlled environments or from a small number of contributors, may miss real-world variability and introduce biases. Unlike these datasets, the RSBdSL38 dataset, free from such limitations, is better suited for comprehensive and accurate BdSL recognition. The diverse participant pool and larger number of images in RSBdSL38 make it a robust resource for developing and testing advanced sign language recognition models, ensuring better generalization and applicability in real-world scenarios. The comparison is shown in the Table 5.13.

Table 5.13: Comparison of existing datasets with RSBdSL38.

Dataset	**Dataset Size**	**No. of Classes**	**Participants**
BDSL49 [10]	29,490	49	14
Shongket [11]	5,820	46	-
BdSL36 [12]	26,713	36	10
KU-BdSL [13]	1,500	30	39
BdSL47 [14]	47,000	47	10
BdSL OPSA22 [16]	33,052	-	7
RSBdSL38	11,864	38	46

5.5.2 Models Comparison and Analysis on RSBdSL38 Dataset

A comparison of the accuracies of various deep learning models applied to a specific dataset is presented here in Figure 5.27 . The x-axis lists the models evaluated, including Xception, VGG16, ResNet50, MobileNetV2, InceptionV3, InceptionResNetV2, DenseNet201, DenseNet121, Ensemble 1, and Ensemble 2, while the y-axis indicates the accuracy percentage. Each bar represents the accuracy of a model, with the accuracy value labeled at the top of each bar for clarity. From the figure, it is evident that Ensemble-1 achieves the highest accuracy at 97.67%, closely followed by ResNet50 with an accuracy of 97.09%. Xception and DenseNet201 also perform well, with accuracies of 97.00% and 97.20%, respectively. The models VGG16, MobileNetV2, InceptionV3, InceptionResNetV2, and DenseNet121 exhibit accuracies in the range of approximately 95.51% to 96.79%, indicating their strong performance but slightly lower compared to the top-performing models. Ensemble-2, while also

an ensemble method, shows a slightly lower accuracy of 95.51% compared to Ensemble-1. The figure highlights the effectiveness of ensemble methods, particularly Ensemble-1, in improving model accuracy. It also showcases the competitive performance of advanced convolutional neural networks like ResNet50 and DenseNet variations, which are well-known for their depth and feature extraction capabilities. This comparison provides valuable insights into the strengths and potential trade-offs of each model, guiding the selection of the most suitable approach for similar datasets and tasks. The Figure 5.28 compares the loss values of various deep learning models on a specific dataset, highlighting DenseNet201 as the top performer with a loss of 0.08, indicating superior error minimization. ResNet50 and

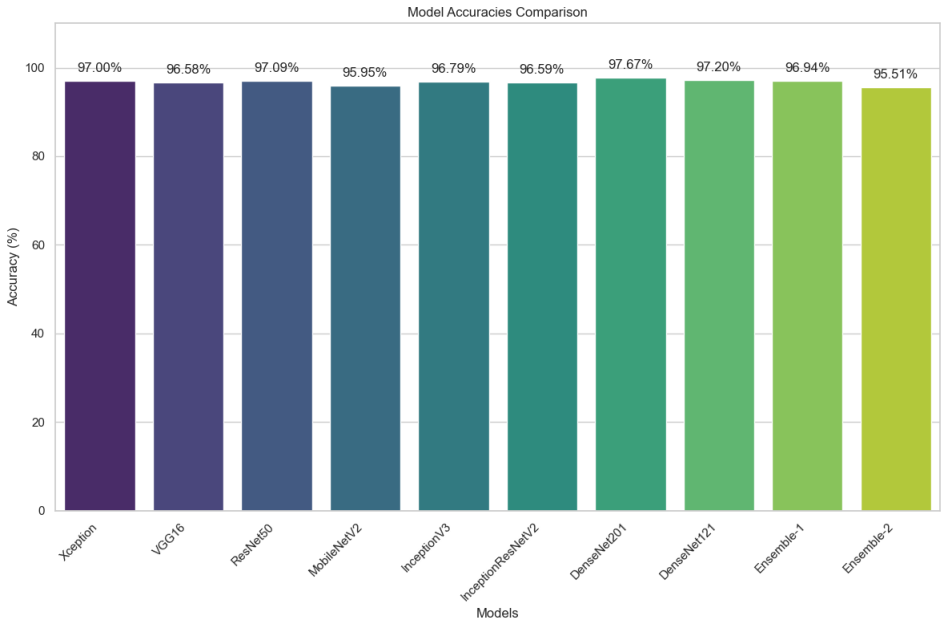


Figure 5.27: Accuracy comparison of various models on RSBdSL38 dataset.

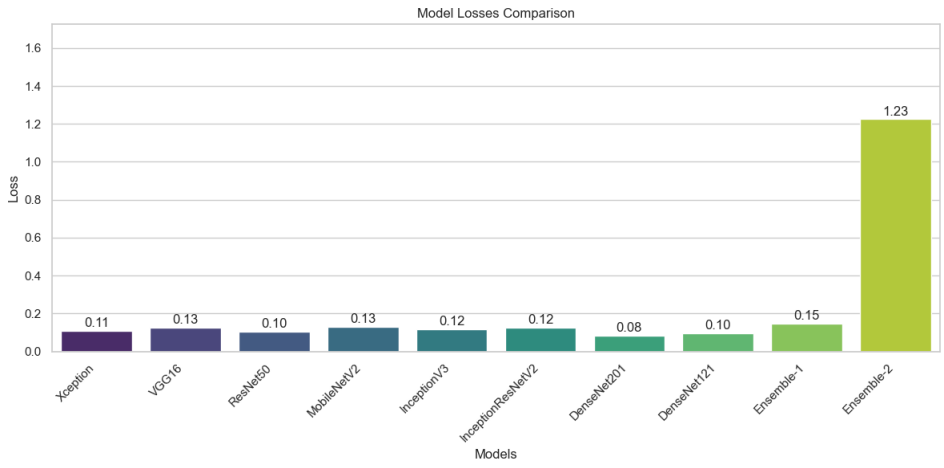


Figure 5.28: Loss comparison of various models on RSBdSL38 dataset.

DenseNet121 follow closely with losses of 0.10 each, demonstrating strong performance in reducing errors. Models like Xception, InceptionV3, and InceptionResNetV2 show slightly higher losses ranging from 0.11 to 0.12, while VGG16 and MobileNetV2 exhibit losses of 0.13, indicating moderate performance in error reduction. Ensemble-1, an ensemble method, achieves a loss of 0.15, potentially acceptable depending on application needs. In contrast, Ensemble-2 performs significantly worse with a loss of 1.23, suggesting issues with its configuration or training data. Overall, DenseNet201 stands out for its effective loss minimization, offering valuable guidance for selecting models aimed at achieving low prediction error.

5.6 Summary

The results and analysis of implemented BdSL recognition models showcase their performance metrics, including accuracy and loss trends across epochs. Detailed evaluations such as loss and accuracy graphs, AUC-ROC analysis, and confusion matrices are presented. The effectiveness of various pretrained models and ensemble techniques in recognizing BdSL gestures is compared, demonstrating their practicality and superiority over existing datasets. This evaluation underscores the efficacy of custom-developed models and ensemble strategies in advancing sign language recognition technology.

CHAPTER 6
Conclusions and Future Works

6.1 Conclusions

The thesis delved into the development and evaluation of deep learning models tailored for Bangla Sign Language (BdSL) recognition, with the primary goal of bridging communication barriers for the hearing-impaired community in Bangladesh. An integral aspect of this research was the meticulous creation of a robust and diverse dataset that accurately captured the nuanced gestures of BdSL, effectively surpassing the limitations of existing datasets. Subsequently, various pre-trained convolutional neural network (CNN) architectures, including Xception, VGG16, ResNet50, MobileNetV2, InceptionV3, Inception-ResNetV2, DenseNet121, and DenseNet201, were implemented and fine-tuned. Through rigorous training and meticulous evaluation, these models demonstrated impressive accuracy in deciphering BdSL gestures, with DenseNet201 emerging as particularly effective. Moreover, the research advanced by exploring ensemble techniques, which amalgamated the strengths of individual models to achieve heightened performance levels. Comparative analyses underscored the superiority of the models developed in this study over existing solutions, especially in contexts where variability in data poses significant challenges. The introduction of ensemble strategies, such as the Xception Ensemble and FusionNet, underscored the potential of collaborative model architectures to elevate accuracy and reliability in BdSL recognition tasks. This research not only furnishes a technological breakthrough for BdSL recognition but also establishes a cornerstone for future innovations in this specialized domain. Emphasizing the pivotal role of comprehensive and diverse datasets, the efficacy of deep learning models in decoding intricate gestures, and the transformative impact of ensemble techniques on enhancing model robustness, this study represents a significant stride forward in the field of sign language recognition. Ultimately, it offers a pragmatic tool for fostering improved communication and inclusivity for the hearing-impaired community, both in Bangladesh and globally.

6.2 Future Recommendations

Building on the findings of this thesis, future research should focus on expanding the dataset to include more gestures and variations, ensuring even greater robustness and applicability. Additionally, exploring advanced deep learning techniques, such as transformers and attention mechanisms, could further enhance the performance of BdSL recognition models. Integrating multimodal data, such as video sequences and depth information, may also provide a richer context for gesture recognition, improving the system's accuracy and usability in real-time applications. Collaborations with linguistic experts and the hearing-impaired community will be essential in refining and validating these models, ensuring they meet the practical needs and expectations of the end-users.

REFERENCES

[1] Y. Ye, Y. Tian, M. Huenerfauth, and J. Liu, "Recognizing american sign language gestures from within continuous videos," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2018, pp. 2064–2073.

[2] M. S. Islam, S. S. S. Mousumi, N. A. Jessan, A. S. A. Rabby, and S. A. Hossain, "Ishara-lipi: The first complete multipurposeopen access dataset of isolated characters for bangla sign language," in *2018 International Conference on Bangla Speech and Language Processing (ICBSLP)*. IEEE, 2018, pp. 1–4.

[3] T. Abedin, K. S. Prottoy, A. Moshruba, and S. B. Hakim, "Bangla sign language recognition using a concatenated bdsl network," in *Computer Vision and Image Analysis for Industry 4.0*. Chapman and Hall/CRC, 2023, pp. 76–86.

[4] A. Núñez-Marcos, O. Perez-de Viñaspre, and G. Labaka, "A survey on sign language machine translation," *Expert Systems with Applications*, vol. 213, p. 118993, 2023.

[5] M. Boháček and M. Hrúz, "Sign pose-based transformer for word-level sign language recognition," in *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, 2022, pp. 182–191.

[6] K. Tarafder, N. Akhtar, M. Zaman, M. Rasel, M. Bhuiyan, and P. Datta, "Disabling hearing impairment in the bangladeshi population," *The Journal of Laryngology & Otology*, vol. 129, no. 2, pp. 126–135, 2015.

[7] S. Siddique, S. Islam, E. E. Neon, T. Sabbir, I. T. Naheen, and R. Khan, "Deep learning-based bangla sign language detection with an edge device," *Intelligent Systems with Applications*, vol. 18, p. 200224, 2023.

[8] S. Das, M. S. Imtiaz, N. H. Neom, N. Siddique, and H. Wang, "A hybrid approach for bangla sign language recognition using deep transfer learning model with random forest classifier," *Expert Systems with Applications*, vol. 213, p. 118914, 2023.

[9] S. Ahmed, "Thesis on bangla sign language detection," https://github.com/baustahmed/thesis, 2024.

[10] A. Hasib, J. F. Eva, S. S. Khan, M. N. Khatun, A. Haque, N. Shahrin, R. Rahman, H. Murad, M. R. Islam, and M. R. Hussein, "Bdsl 49: A comprehensive dataset of bangla sign language," *Data in Brief*, vol. 49, p. 109329, 2023.

[11] S. N. Hasan, M. J. Hasan, and K. S. Alam, "Shongket: A comprehensive and multipurpose dataset for bangla sign language detection," in *2021 International Conference on Electronics, Communications and Information Technology (ICECIT)*. IEEE, 2021, pp. 1–4.

[12] O. B. Hoque, M. I. Jubair, A.-F. Akash, and S. Islam, "Bdsl36: A dataset for bangladeshi sign letters recognition," in *Proceedings of the Asian Conference on Computer Vision*, 2020.

[13] A. A. J. Jim, I. Rafi, M. Z. Akon, U. Biswas, and A.-A. Nahid, "Ku-bdsl: An open dataset for bengali sign language recognition," *Data in Brief*, vol. 51, p. 109797, 2023.

[14] S. Rayeed, S. T. Tuba, H. Mahmud, M. H. U. M. Md, S. H. M. Md, and K. H. Md, "Bdsl47: A complete depth-based bangla sign alphabet and digit dataset," *Data in Brief*, vol. 51, p. 109799, 2023.

[15] M. M. Islam, M. R. Uddin, M. J. Ferdous, S. Akter, and M. N. Akhtar, "Bdslw-11: Dataset of bangladeshi sign language words for recognizing 11 daily useful bdsl words," *Data in Brief*, vol. 45, p. 108747, 2022.

[16] M. A. Rahaman, K. U. Oyshe, P. K. Chowdhury, T. Debnath, A. Rahman, and M. S. I. Khan, "Computer vision-based six layered convneural network to recognize sign language for both numeral and alphabet signs," *Biomimetic Intelligence and Robotics*, vol. 4, no. 1, p. 100141, 2024.

[17] "Proyash, rangpur (a school of special education)," https://proyashrangpur.edu.bd.

[18] "Mymensingh welfare school, mymensingh." https://www.mwsbd.org/.

[19] "Sand autism school, gazipur." https://www.facebook.com/SandAutismSchoolGazipur.

[20] Wikipedia contributors, "Lanczos resampling — Wikipedia, the free encyclopedia," 2024, [Online; accessed 11-June-2024]. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Lanczos_resampling&oldid=1225247839

[21] S. G. K. Patro and K. K. Sahu, "Normalization: A preprocessing stage," 2015.

[22] S. Sharma, "Data splitting strategies in machine learning," https://www.linkedin.com/pulse/data-splitting-strategies-machine-learning-swapnil-sharma.

[23] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

[24] L. Ali, F. Alnajjar, H. Jassmi, M. Gochoo, W. Khan, and M. Serhani, "Performance evaluation of deep cnn-based crack detection and localization techniques for concrete structures," *Sensors*, vol. 21, p. 1688, 03 2021.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: http://arxiv.org/abs/1512.03385

[26] A. Tragoudaras, P. Stoikos, K. Fanaras, A. Tziouvaras, G. Floros, G. Dimitriou, K. Kolomvatsos, and G. Stamoulis, "Design space exploration of a sparse mobilenetv2 using high-level synthesis and sparse matrix techniques on fpgas," *Sensors*, vol. 22, p. 4318, 06 2022.

[27] Q. Gao, U. Ogenyi, J. Liu, Z. Ju, and H. Liu, *A Two-Stream CNN Framework for American Sign Language Recognition Based on Multimodal Data Fusion*, 01 2020, pp. 107–118.

[28] I. Tareq, B. Elbagoury, S. El-Regaily, and E.-S. El-Horbaty, "Analysis of ton-iot, unw-nb15, and edge-iiot datasets using dl in cybersecurity for iot," *Applied Sciences*, vol. 12, p. 9572, 09 2022.

[29] Nillmani, N. Sharma, L. Saba, N. N. Khanna, M. K. Kalra, M. M. Fouda, and J. S. Suri, "Segmentation-based classification deep learning model embedded with explainable ai for covid-19 detection in chest x-ray scans," *Diagnostics*, vol. 12, no. 9, 2022. [Online]. Available: https://www.mdpi.com/2075-4418/12/9/2132

[30] M. Hasan, M. Fatemi, M. Khan, M. Kaur, and A. Zaguia, "Comparative analysis of skin cancer (benign vs. malignant) detection using convolutional neural networks," *Journal of Healthcare Engineering*, vol. 2021, pp. 1–17, 12 2021.

APPENDIX A

Raw data processing:

```
import os
from PIL import Image, ExifTags
from tqdm import tqdm

def resize_and_rename_images(source_folder, destination_folder, target_size=(224, 224)):
    os.makedirs(destination_folder, exist_ok=True)

    for subfolder_name in os.listdir(source_folder):
        subfolder_path = os.path.join(source_folder, subfolder_name)

        output_subfolder = os.path.join(destination_folder, subfolder_name)
        os.makedirs(output_subfolder, exist_ok=True)

        image_count = 1

        progress_bar = tqdm(os.listdir(subfolder_path), desc=f'Resizing images in class
            {subfolder_name}')

        for filename in progress_bar:
            file_path = os.path.join(subfolder_path, filename)

            with Image.open(file_path) as img:
                if hasattr(img, '_getexif'):
                    exif = img._getexif()
                    if exif is not None:
                        orientation = exif.get(0x0112)

                        if orientation == 3:
                            img = img.rotate(180, expand=True)
                        elif orientation == 6:
                            img = img.rotate(270, expand=True)
                        elif orientation == 8:
                            img = img.rotate(90, expand=True)

                img_resized = img.resize(target_size, Image.LANCZOS)

                new_filename = f"class-{subfolder_name}-{image_count}.jpg"
                image_count = image_count +1

                output_file_path = os.path.join(output_subfolder, new_filename)
                img_resized.save(output_file_path)

                progress_bar.set_postfix({'File': new_filename})

source_folder_path = "F:/datasets/dataset"
destination_folder_path = "F:/thesis/data"
resize_and_rename_images(source_folder_path, destination_folder_path)
```

Data Loading Function:

```
gpus = tf.config.list_logical_devices('GPU')
stg=tf.distribute.MirroredStrategy(gpus)

folder_dir = location
total_files = sum(len(files) for _, _, files in os.walk(folder_dir))

```

```
with tqdm(total=total_files, desc="Processing Images") as pbar:
    for folder in os.listdir(folder_dir):
        for file in os.listdir(os.path.join(folder_dir, folder)):
            image_path = os.path.join(folder_dir, folder, file)
            img = cv2.imread(image_path)
            img_resized = cv2.resize(img, (size, size))
            cv2.imwrite(image_path, img_resized)
            pbar.update(1)
picture_size = (size, size)
train_set = tf.keras.utils.image_dataset_from_directory(
    directory=folder_dir,
    shuffle=True,
    image_size=picture_size,
    batch_size=batch_size,
    label_mode='categorical',
    validation_split=0.2,
    subset='training',
    seed=22
)

validation_set = tf.keras.utils.image_dataset_from_directory(
    directory=folder_dir,
    shuffle=True,
    image_size=picture_size,
    batch_size=batch_size,
    label_mode='categorical',
    validation_split=0.2,
    subset='validation',
    seed=22
)
```

Data Augmentation:

```
img_augmentation = tf.keras.Sequential([
        tf.keras.layers.experimental.preprocessing.RandomRotation(factor=0.2),
        tf.keras.layers.experimental.preprocessing.RandomTranslation(
            height_factor=0.1, width_factor=0.1),
        tf.keras.layers.experimental.preprocessing.RandomZoom(
            height_factor=0.2),
        tf.keras.layers.experimental.preprocessing.RandomFlip(
            mode='horizontal'),
        tf.keras.layers.experimental.preprocessing.RandomContrast(factor=0.2)
    ], name="img_augmentation")
```

Model Training:

```
lr_scheduler = ReduceLROnPlateau(monitor='val_loss',
                                 factor=0.1,
                                 patience=2,
                                 verbose=1)
early_stopping = EarlyStopping(monitor='val_loss',
                               patience=5,
                               verbose=1)

callbacks = [lr_scheduler, early_stopping]
history = model.fit(train_set, epochs=100, validation_data=validation_set, callbacks=
    callbacks,
                    steps_per_epoch=len(train_set), validation_steps=len(validation_set)
                        )
```

Model Evaluation:

```
ensemble_predictions = []
num_models = 3
test_data_dir = location
test_datagen = ImageDataGenerator()
test_generator = test_datagen.flow_from_directory(
    test_data_dir,
    target_size=picture_size,
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)
# Generate predictions for each model and collect them in a list
for model in [model_1, model_2, model_3]:
    predictions = model.predict_generator(test_generator)
    ensemble_predictions.append(predictions)

# Compute the average predictions of the ensemble
ensemble_predictions = np.mean(ensemble_predictions, axis=0)

# Calculate the accuracy of the ensemble predictions
ensemble_acc = np.mean(np.argmax(ensemble_predictions, axis=1) == test_generator.classes
    )

print('Ensemble accuracy:', ensemble_acc)
```

APPENDIX B

Complex Engineering Problems (CP) and Complex Engineering Activities (CA) Analysis

Title: Bangla Sign Language Recognition: Dataset Innovation and Methodologies.

Attainment of Complex Engineering Problem (CP)

S.L.	CP No.	Attainment	Remarks
1.	P1: Depth of Knowledge Required	Yes	K3 (Engineering Fundamentals): python describe in Chapter 4 Art No: 4.2.2.
			K4 (Engineering Specialization): knowledge about Deep learning describe in Chapter 3. Art No: 3.3
			K5 (Design): Methodology flow chart describe in Chapter 3 Art. No: 3.5.
			K6 (Technology): Tensorflow, Numpy, Pandas, etc. Described on Chapter 4 Art. No: 4.2.
			K8 (Research): Related work describe in Chapter 2
2.	P2: Range of Conflicting Requirements	Yes	Learn about Bangla Sign Language describe in Chapter 1 Art. No: 1.2 and knowledge about deep learning describe in Chapter 3. Art No: 3.3.

3.	P3: Depth of Analysis Required	Yes	Build new dataset about Bangla Sign Language described in 3 Art. No: 3.2.
4.	P4: Familiarity of Issues	Yes	Study about Bangla Sign Language as a CSE students describe in Chapter 1 and Chapter 3.
5.	P5: Extent of Applicable Codes	Yes	Follow standards methodology describe on Chapter 3
6.	P6: Extent of Stakeholder Involvement and Conflicting Requirements	Yes	Involves Post Office, All of the people who are involved are digital Documents processing Chapter 1 Art. No: 1.6
7.	P7: Interdependence	Yes	Collecting Dataset describe in Chapter 3 Art. No: 3.2. Apply different augmentations also describe in Chapter 3 Art. No: 3.2.

Mapping of Complex Engineering Activities (CA)

S.L.	**CA No.**	**Attainment**	**Remarks**
1.	A1: Range of resources	Yes	Involves Post Office, All of the people who are involved are digital Documents processing Chapter 1 Art. No: 1.6
2.	A2: Level of interaction	Yes	There are several issues come when we work this Thesis describe in Chapter 1 Art. No: 1.5
3.	A3: Innovation	Yes	Build new dataset and build model for detect Bangla Sign Language in Chapter 3 and chapter 4.
4.	A4: Consequences for Society and the Environment	Yes	Involves Schools, All of the people who are involved are mute and deaf Chapter 1 Art. No: 1.4.

5.	A5: Familiarity	Yes	Build a new Bangla Sign Language Dataset and recognition model describe in Chapter 3 and chapter 4.

BIOGRAPHY

of

Saad Ahmed

Saad Ahmed, born to Md. Shahjahan and Mst. Manjuara Khatun, is a dedicated Computer Science researcher currently pursuing a B.Sc. in Computer Science and Engineering at Bangladesh Army University of Science and Technology (BAUST), Saidpur. His educational foundation includes a perfect GPA of 5.00 in SSC and 4.26 in HSC from Agricultural University High School, Mymensingh. Saad excels in Machine Learning, Deep Learning, Computer Vision, and Image Processing, notably developing the ”Primary School Management” system for Chatrapur Primary School, Saidpur. He is skilled in MVC architectures for web development. Saad's leadership is evident in his roles as an executive member of the BAUST CSE Society and Career Society, demonstrating strong organizational skills. He is passionate about learning technologies and making a positive impact in Computer Science and Engineering.

Thesis:

- **Title:** Bangla Sign Language Recognition: Dataset Innovation and Methodologies.

Saad Ahmed
+8801724407189
saad.ahmed.arst@gmail.com
Mymensingh Sadar, Mymensingh

BIOGRAPHY

of

Razia Sultana

Razia Sultana, born in Jamalpur, daughter of Md. Riaz Uddin and mst. Anjuman Ara Khatun, who is a dedicated Computer Science enthusiast and researcher. She is pursuing her B.Sc. Engineering in Computer Science and Engineering from Bangladesh Army University of Science and Technology, Saidpur. She completed her SSC from Agricultural University High School, Mymensingh with GPA 5.00 and HSC from Queen's School and College, Dhaka with GPA 4.44. In academics, Razia Sultana has demonstrated her commitment to advancing knowledge through her research endeavors. She is actively engaged in research in the field of Machine Learning and Deep learning. Razia played an important work in her thesis research Bangla Sign Language Classification.

Thesis:

- **Title:** Bangla Sign Language Recognition: Dataset Innovation and Methodologies.

Razia Sultana
+8801768837211
razia.sultana.arst@gmail.com
Gazipur Sadar, Gazipur